

——— // REWRITING THE DESKTOP // ———

The GUI Automation BUS

First Edition

THE BUS

the surface · the bridge · the socket

2026-07-18

MenkeTechnologies

Contents

Before You Begin	3
Chapter 1 — The Dead End	5
Chapter 2 — Five Layers, One Contract	7
Chapter 3 — The Surface	9
Chapter 4 — The Bridge	12
Chapter 5 — The App Module	14
Chapter 6 — The Transport	16
Chapter 7 — The Hybrid Handler	19
Chapter 8 — The Surface, App by App	21
Chapter 9 — Three Front-Ends, One Module	24
Chapter 10 — Security and the Allow-List	26
Chapter 11 — Novelty — An Honest Prior-Art Analysis	28
Chapter 12 — Rollout and Open Questions	30
Chapter 13 — The Value Model at the Edge	32
Chapter 14 — Failure Is a Reply	34
Chapter 15 — Order, Correlation, and the Pump	36
Chapter 16 — Orchestrations	38
Chapter 17 — The Surface as Its Own Documentation	43
Chapter 18 — The Long Arc	45
Chapter 19 — Writing a New App Module	47
Chapter 20 — The Subscription and Event Model	50
Chapter 21 — Testing the Bus	52
Chapter 22 — Versioning the Contract	54
Chapter 23 — Debugging and Tracing Requests	56
Chapter 24 — Performance and Backpressure	58

Chapter 25 — A Per-App Orchestration Cookbook	60
Chapter 26 — The Verb Reference	64
Chapter 27 — State Queries in Depth	71
Chapter 28 — The Frame Reference	73
Chapter 29 — The Error Reference	75
Chapter 30 — Cookbook II: Reporting, Maintenance, Migration	77
Chapter 31 — Implementing the Handler	81
Chapter 32 — Cookbook III: Interactive and Composed Sessions	83
Chapter 33 — The Bus and the Connector Family	85
Chapter 34 — Patterns and Anti-Patterns	87
Chapter 35 — Cookbook IV: Domain Recipes	89
Chapter 36 — A Design FAQ	93
Chapter 37 — Glossary	96
Chapter 38 — Tooling: The REPL and Completion in Practice	98
Chapter 39 — A Complete Session, Narrated	101
Chapter 40 — The Bus in the Shell	104

Before You Begin

This is the book about a single sentence: **one language should be able to drive every app you own, by name, and get an answer back.**

Not by moving the mouse. Not by matching pixels. Not by shelling out to a different scripting runtime for each platform. By calling a named verb — a verb the app itself declares — passing typed arguments, and receiving the return value straight into the calling program's variables. `library.search` with a query returns rows. `request.send` returns a response. `document.open` returns a handle. The app is not a screen to be poked; it is a set of functions to be called.

The suite this book describes was built the way described in its companion, *Rewriting the Desktop in Rust*: every app is a thin Tauri v2 shell over a pure-Rust `*-core` engine, all sharing one component library, `zgui-core`. That architecture left a gift on the table. If each engine already exposes a clean command surface to its own shell, then that same surface — enumerated, typed, and reachable over a small local protocol — is an automation API for free. The GUI Automation Bus is the collection of the small pieces that turn “the shell can call the engine” into “any stryke script, anywhere on the machine, can call the engine, in-process or over a socket, and orchestrate a dozen apps in one run.”

The language is stryke — the author's own from-scratch, Cranelift-JIT scripting language. The bus is not a new language and not a network service. It is an `App` module for stryke, a typed surface declared once per app, a request/response bridge, and a per-app Unix socket carrying newline-delimited JSON. Five small layers, each with a reference implementation already present somewhere in the stack, assembled into a capability the individual pieces did not have alone.

How to read this book

The first six chapters build the bus from the bottom up: the problem it closes (Chapter 1), the five-layer model (Chapter 2), then each layer in turn — the surface (3), the bridge (4), the `App` module (5), and the transport (6). Chapter 7 reads the real hybrid handler that every app ships. Chapter 8 is the census: the live surface, app by app — more than three thousand verbs across eleven apps. Chapters 9 through 12 cover the three front-ends onto the bus, the security model, an honest accounting of what is and is not novel here, and the rollout.

The remaining chapters turn from the contract to the craft. Chapters 13 through 15 go down to the seams the earlier chapters glide over — how a JSON value becomes a stryke value and back, how a failure becomes a reply, and how correlation, ordering, and event delivery actually work on a full-duplex socket. Chapter 16 is worked orchestrations: whole cross-app scripts, read line by line, that exist only because the bus does. Chapter 17 shows the surface documenting itself, and Chapter 18 traces the arc past version one. The first half earns the design; the second half lives inside it.

The last stretch is the practitioner's half — the material you reach for once the design is settled and you are building on top of it. Chapter 19 is a full tutorial: take an app that has no surface and put it on the bus, one line at a time, from the first `register` call to a passing `.stk` script. Chapter 20 is the subscription and event model in depth — filters, replay, backpressure on the stream, and the lifecycle of a subscription from `sub` frame to `unsub`. Chapter 21 is how you test a bus surface without a running desktop; Chapter 22 is how the contract changes without breaking the scripts already written against it; Chapter 23 is what you do when a call misbehaves — tracing, logging, and reading the wire. Chapter 24 is performance and backpressure under real load. Chapter 25 is a cookbook: a run of worked, per-app orchestrations organized by the problem they solve rather than the mechanism they use. Chapter 26 is the Verb Reference — a namespace-by-namespace walk of every app's surface, the printed companion to the generated catalog. The final four are reference matter for

the desk: Chapter 27 is state queries in depth — when to read state and when to call a verb; Chapter 28 is the frame reference, every wire frame with its fields and a worked transcript; Chapter 29 is the error reference, every failure the bus can hand a script with its cause and its fix; and Chapter 30 is a second cookbook of reporting, maintenance, and migration runs. Chapter 31 goes to the app author's side — the `zgui-bridge` crate, the `Handler`, and what `serve` actually does. Chapter 32 is a third cookbook of interactive and composed sessions, and Chapter 33 places the `App` module in the connector family it belongs to, alongside `stryke-gui` and `stryke-aws`. Chapters 34 and 35 collect the idioms and a fourth cookbook of single-app recipes; Chapter 36 is a design FAQ that answers the questions the earlier chapters raise but do not stop for; Chapter 37 is a glossary of every load-bearing term; and Chapter 38 builds the REPL and the shell completion far enough to use. Read the first eighteen chapters to understand the bus; read the rest to live in it day to day.

Every architectural claim is grounded in the design document `GUI_AUTOMATION_BUS.md` and in the shipped code — each app's `bus.rs`, each engine's command list, the shared `zgui-core` surface registry. Where a number appears — a verb count, an app count — it is derived from the source of truth (`GUI_SCRIPT_ACTIONS.md` is generated from each app's verb list), not asserted from memory. This is a book about a built thing, and it tries to earn every sentence.

Chapter 1 – The Dead End

A palette that cannot talk back

Before the bus, every app already shipped a user-programmable command palette. `ZGui.userCommands`, in `zgui-core/webui/user-commands.js`, stores a shared, cross-app list of command entries, each a chain of typed steps drawn from a small grammar: `url`, `js`, `scheme`, `event`, `stryke`, `bash`. Two of those steps are the whole story of why the bus had to exist.

The `event` step dispatches exactly one app action by id. Apps published their action vocabulary through `setActions([{id, label}])` — id and label, nothing more. No parameters. No return value. No way to read the app's state first, or to react to what the action did. It is a doorbell: you press it, something happens inside, and you are told nothing.

The `stryke` step ran a script on the backend through `run_stryke_hook`. The script received a single context argument and returned. That was the entire contract. The script was blind: it could compute, it could format a string, it could call out to the wider world — but it had **no handle back into the app it was running inside**. It could not call an app action and read the result. It could not query the current selection, the open document, the contents of a list. It could not subscribe to an event. And it certainly could not reach a *different* app.

So the palette offered a cruel choice. Either a blind script that could not touch the app, or a single fire-and-forget action that returned nothing. Never the obvious thing in between: a script that drives the app, waits for the answer, and decides what to do next. That gap — the missing return path from app to script — is the entire problem the bus closes.

What “driving” actually requires

Reduce the requirement to its verbs and it is short. A running script must be able to **enumerate** an app's actions, **call** one with typed arguments and receive the **return value**, **read** a named piece of state, and **subscribe** to an event stream. Four operations. And it must be able to do all four whether the script runs *inside* the app — a palette step, a hook — or *outside* it, as a standalone file in the shell driving a separately running app.

Those two execution contexts look different and are the same. In-process, the script and the app share an address space; the call is a function call. Out-of-process, they are two operating-system processes and the call must cross a socket. The bus's central design decision is that the script cannot tell which it is in. `App::here()->call(...)` and `App::open("zcite")->call(...)` return handles that behave identically. One language, one API, two transports underneath.

Why the existing steps could not be stretched

It is fair to ask why the palette's other step kinds could not simply be pressed into the role, since the grammar already had six. Walk them and each fails the same test for a different reason. The `url` step opens a location; it is a navigation primitive with no return channel and no notion of an app action at all. The `scheme` step fires a custom-scheme URL — `zcite://item/42` — which is one-way by construction: a scheme handler is invoked, not queried, and the sender learns nothing about what the handler did. The `js` step evaluates JavaScript in the front-most webview, which *can* read app state, but only the app the palette is attached to, only

in the frontend, and only by writing app-specific JavaScript that reaches into private internals — the opposite of a stable, typed, cross-app contract. The `bash` step shells out, which is the widest escape hatch of all and precisely the thing the whole suite was built to stop needing: a bash step cannot call an app verb because there is no command-line surface for the app's verbs to call. Six steps, and not one of them could read a value out of an app and hand it to the next step. The gap was not a missing feature in any single step; it was a missing *layer* that all the steps could sit on top of.

The four operations, made concrete

The four operations read as abstractions — enumerate, call, read, subscribe — so it is worth grounding each in the single line of stryke that will, by Chapter 5, express it. *Enumerate* is `$h->verbs()`, which returns the typed manifest a script reads to discover what an app can do. *Call* is `$h->call("item.add", %{ doi => $d })`, which invokes a verb with typed arguments and — the entire point — binds the return value into a stryke variable. *Read* is `$h->get("selection")`, which pulls a named piece of live state without disturbing it. *Subscribe* is `$h->on("itemAdded", $fn)`, which registers a callback against an event stream. Four verbs of the meta-language, and every automation in this book is built from nothing but their composition. The claim of the chapter is narrow and total at once: before the bus, a script inside an app could do none of the four against the app it lived in, and a script outside could do none of the four against anything. After the bus, a script does all four, from either side, through one handle. That is the whole distance the rest of the book travels.

Chapter 2 – Five Layers, One Contract

The stack, top to bottom

The bus is five layers, and the discipline of the design is that each one already had a working precedent in the stack — nothing here is invented where something could be reused.

Layer 1, the automation surface. Each app declares a typed dictionary of verbs, state queries, and events. This is the formalization of `setActions`: the same idea, upgraded from id-and-label to id, parameters, return type, and a runnable body. It is owned by the app’s frontend and registered once on boot.

Layer 2, the bridge. A request/response mechanism so that a stryke `call` returns a value from the app back into the VM. The reference implementation is zcontainer’s “sync invoke plus streaming subscribe” substrate — the exact shape the bus needs, ported into a shared component rather than rebuilt per app.

Layer 3, the stryke App module. A connector package in the same family as `stryke-gui` and `stryke-aws`: a cdylib with C-ABI exports and a stryke-facing module. It gives the language `App::open`, `App::here`, `App::list`, and the handle methods `verbs`, `call`, `get`, and `on`.

Layer 4, the transport. In-process, a direct host callback — no socket, lowest latency. Out-of-process, a per-app Unix domain socket speaking newline-delimited JSON. The precedent is zterminal, which already talks a raw wire protocol straight to a socket — the tmux imsg protocol, no subprocess — proving the discipline.

Layer 5, the front-ends. The palette `stryke` step, the hooks runtime, and the standalone `.stk` file. All three get `App` in scope and use the same module, so there is no per-surface special-casing. The palette does not become a client of a special palette API; it becomes one client of the same bus every other surface uses.

The ownership rule

One rule governs where each piece lives, and it is inherited directly from the architecture book’s shell-versus-engine boundary: **the automation surface is owned by the core, and the socket host is owned by the top-level app.** A core knows its own verbs — it is the thing that implements them — so the verb list is core-owned. But a running app is one process with one front door, the way it has one command palette; so it opens exactly one socket, and any engine embedded inside it contributes its verbs *into that app’s surface* rather than opening a socket of its own. Namespacing keeps them distinct. The rule falls straight out of the existing architecture; the bus did not have to invent a new notion of who owns what.

Each layer’s precedent, named

The design’s central discipline — reuse a working shape rather than invent one — is worth making explicit layer by layer, because it is what keeps the per-app cost near zero and what makes the whole thing credible as a *built* rather than *proposed* system. Layer 1 is `setActions` grown a type signature: the registration call was already there, already invoked on boot in every app, already read by the palette’s action picker; the bus widens its payload and adds two readers. Layer 2 is zcontainer’s invoke-plus-subscribe substrate, a request/response channel with an attached stream that was written, shipped, and debugged before the bus existed as a word. Layer 3 is the `stryke-gui` / `stryke-aws` connector pattern — a cdylib with C-ABI exports and a stryke-facing module — instantiated a third time with different verbs. Layer 4 is zterminal’s socket discipline: a real app already speaks a raw wire protocol straight to a Unix socket with no subprocess between, so the bus is not

proving that the pattern works, only pointing a second protocol down the same proven pipe. Layer 5 is the palette's existing step runner, which already put stryke in scope on three surfaces; the bus adds one binding to that scope and changes nothing else. Five layers, five precedents. The novelty, as Chapter 11 argues at length, is the assembly, not any single course of brick.

The contract, stated once

Underneath the five layers is one contract, and it can be written in a sentence that every layer either implements or consumes: *a caller names a verb and supplies typed arguments; the app runs the verb and returns exactly one typed value, or one typed failure, correlated to the call.* Layer 1 declares the verbs and their types. Layer 2 carries the call and its reply. Layer 3 gives the caller a handle to name the app and the verb. Layer 4 moves the bytes, in-process or across a socket, without changing the contract's shape. Layer 5 is three different callers all speaking that one sentence. Reading the stack this way explains why the transport can be swapped under the caller's feet without the caller noticing: the transport is the one layer the contract deliberately says nothing about. A verb call means the same thing whether a host callback or a socket carried it, because the contract is defined in terms of names, types, and correlation — never in terms of wires.

Chapter 3 – The Surface

From label to manifest

Layer 1 replaces `setActions([ {id, label} ])` with a typed manifest registered once per app, through a `zgui-core` module, `automation.js`, exposed as `window.ZGui.automation`. The registration is declarative:

```
ZGui.automation.register({
  app: "zcite",
  verbs: [
    { id: "library.search",
      label: "Search library",
      params: [{ name: "q", type: "string", required: true },
                { name: "collection", type: "string", required: false }],
      returns: "list<item>",
      run: (a) => zcite.searchLibrary(a.q, a.collection) },
    { id: "item.add",
      label: "Add item",
      params: [{ name: "doi", type: "string", required: true }],
      returns: "item",
      run: (a) => zcite.addByDoi(a.doi) },
  ],
  state: [
    { id: "selection", returns: "list<item>", get: () => zcite.currentSelection() },
    { id: "activeCollection", returns: "string", get: () => zcite.activeCollection() },
  ],
  events: [
    { id: "itemAdded", payload: "item" },
  ],
});
```

Three kinds of entry. A **verb** is callable and typed and returns a value; its `run` may be asynchronous, and the bridge awaits it before replying. A **state** query is read-only; its `get` returns a value on demand. An **event** is a name the app emits, via `ZGui.automation.emit(id, payload)`, and that subscribers receive.

Introspection is the same surface

`ZGui.automation.surface()` returns the manifest with the function bodies stripped — types only, no `run` or `get` closures. That stripped manifest is exactly what a script sees when it asks a handle for its `verbs()`. The consequence is that the palette's own action-picker, which used to read the flat id-and-label list, now reads the typed surface for free: the same registration that makes an app scriptable from the outside makes its palette editor richer on the inside. One declaration, two readers.

What the types are, and what they are not

The `type` and `returns` strings in the manifest — `string`, `list<item>`, `item` — look like a type system, and it is worth being precise about how much of one they are. They are a **contract the bridge checks at the boundary and a document the surface publishes**, not a compiler that has proven anything about the `run` body. When a call arrives, the bridge validates the supplied arguments against the declared `params`: a missing `required` argument is rejected before `run` is ever invoked, and a value of the wrong JSON kind is rejected with the verb's signature in the error. That much is enforced. But `returns: "list<item>"` is a promise the app makes about its own output, not a shape the bridge coerces the result into — if a verb's `run` returns something else, the bridge forwards it faithfully and the mismatch is the app's bug to fix, surfaced the next time a script reads a field that is not there.

The choice is deliberate and it mirrors the whole suite's posture toward its own frontend: the core is trusted because the same author wrote it, so the manifest's job is to describe the surface accurately and reject *callers* who misuse it, not to defend the app against itself. The `item` in `list<item>` is documentation with teeth only at its edges — it tells a script author what fields to expect and it lets `verbs()` render a readable signature, while the interior of the value is governed by the app's own discipline. A richer scheme — nominal record types shared between the manifest and the engine — is a thing the design leaves for a later edition, named in Chapter 18, precisely because the cheap version already carries its weight.

The parameter grammar in full

The `params` array is where the surface earns its keep, so it repays a closer read than the opening example gave it. Each entry is an object with a `name`, a `type`, and a `required` flag, and the bridge treats the three as a checklist it runs before `run` is ever entered. A missing `required` parameter is a rejection, not a call with a nil hole in it. An `optional` parameter absent from the arguments is simply not passed, and the `run` body sees it as undefined — the app decides its own default. The `type` string is drawn from the same small vocabulary the `returns` field uses: the scalars `string`, `int`, `float`, `bool`; the container forms `list<T>` and `map<T>`; and the app's own nominal shapes such as `item`, `message`, `transfer`, which the surface treats as opaque object types with named fields the app documents. A verb that takes a search query and an optional collection reads:

```
{ id: "library.search",
  label: "Search library",
  params: [{ name: "q", type: "string", required: true },
            { name: "collection", type: "string", required: false },
            { name: "limit", type: "int", required: false }],
  returns: "list<item>",
  run: async (a) => zcite.searchlibrary(a.q, a.collection, a.limit ?? 50) }
```

Two properties of that entry are load-bearing. First, `run` is `async` and returns a Promise, and the bridge awaits it before composing the reply — so a verb backed by a database query or a network fetch reports its real result, not a placeholder, and the calling script blocks exactly as long as the work takes. Second, the default `a.limit ?? 50` lives in the `run` body, not in the manifest: the manifest declares that `limit` is an optional int, and the app decides what absence means. The surface describes the shape of the conversation; the engine owns the semantics.

Emitting an event

The third entry kind, the event, is the only one whose direction is app-to-script, and its declaration is the smallest of the three — a name and a payload type — but its runtime side has a second half the registration does not show. Declaring `{ id: "itemAdded", payload: "item" }` only names the channel; the app must actually push down it, which it does with one call at the point the thing happens:

```
async function addByDoi(doi) {  
  const item = await zcite.resolveAndStore(doi);  
  ZGui.automation.emit("itemAdded", item); // every subscriber receives this  
  return item;  
}
```

The `emit` is fire-and-forget from the app's side — it returns immediately, having handed the payload to the bridge, which fans it out to every subscription registered on that event. An app with no subscribers still calls `emit`; the bridge simply has no one to deliver to, and the cost is a membership check on an empty list. This is why an app can declare and emit events unconditionally without paying for machinery no script is using: the expensive half of a subscription — the delivery — exists only for events someone is actually watching, and Chapter 20 traces exactly how the bridge decides who that is.

Chapter 4 – The Bridge

Making a fire-and-forget call return

Layer 2 is the piece that turns `run_stryke_hook` from a dead end into a conversation. The bridge lets an in-flight stryke script call back into the app and — the whole point — get the value back into the VM. Trace a single in-process `call`:

```
stryke VM (App::call)
-> host callback app_call(verb, args_json)      [stryke FFI -> app backend]
-> backend routes to the webview: dispatch to the registered verb
-> verb.run(args) resolves (await if it returned a Promise)
-> JSON result travels back over the response channel
-> host callback returns the JSON to the stryke VM
-> App::call returns the decoded value
```

The one genuinely new backend primitive is a **synchronous-looking request/ response** between the stryke host and the webview surface: the script blocks on a call, the webview runs the verb, and the value comes back correlated to that call. This is not invented from nothing. `zcontainer` already implements exactly this shape — a synchronous `invoke` plus a streaming `subscribe` — and the bus's design is to lift that substrate into a shared component, not to reimplement it in each app.

One substrate, two channels

`get` travels the same request/response path, aimed at the state table instead of the verb table. Subscription is the streaming half of the same substrate: `on(event, fn)` registers on the subscribe channel, and the app's `emit` pushes frames to every subscriber. So the bridge is not three mechanisms but one — a correlated request/response with an attached event stream — used three ways. That economy is why per-app cost stays near zero: an app that wires the one substrate gets calls, state reads, and subscriptions together.

The correlation channel, concretely

The word “correlated” carries the weight in that trace, and it is worth making mechanical. When the host callback delivers a call into the backend, it assigns the request a numeric `id` and stashes a one-shot reply slot in a table keyed by that `id` — in the shipped code, a channel the backend can later fulfill exactly once. The backend then evaluates the dispatch shim in the webview, `window.__zguiBridgeDispatch(id, kind, name, args)`, passing the same `id` along. The shim looks up the registered verb by `name`, runs it, awaits the result, and calls back into the backend through a small Tauri command, `zgui_bridge_reply(id, ok, value)`. The backend takes the `id`, finds the waiting slot, and fulfills it with the `value` — which unblocks the host callback, which returns the JSON to the stryke VM. The `id` is the thread that runs through the whole loop: it is minted at the call, it rides into the webview and back, and it is what lets the reply find the exactly-one caller waiting for it even when several calls are in flight. Nothing about that loop is specific to `call`; `get` runs the identical machinery with `kind` set to state instead of verb, which is the concrete meaning of “one substrate, two channels.”

Why the webview is in the loop at all

A reasonable objection is that routing a call through the webview adds a hop, and the design answers it by *not* routing every call through the webview — which is exactly what the hybrid handler of Chapter 7 is for. Engine verbs, the ones listed in the core's `COMMANDS`, never enter the webview at all: they run in Rust, in the same `*-core` engine the UI calls, and the reply is composed without a single frame crossing into JavaScript. The webview hop exists only for the surface that genuinely lives in the frontend — the `appshell.*` verbs, an embedded core's verbs, anything a plugin registered through `ZGui.automation` at runtime. For those, the frontend is not an overhead in the path; *it is* the path, because the frontend is where the verb was defined. The bridge's two-target shape is therefore not a compromise between speed and reach but a recognition that an app is two things at once — a Rust engine and a web frontend — and a call has to be able to land on whichever one actually implements the verb.

Chapter 5 – The App Module

A connector, not a new dialect

Layer 3 is a stryke connector package, `stryke-app`, a sibling of `stryke-gui` and `stryke-aws` and built to the same cdylib-plus-FFI shape: C-ABI functions taking and returning C strings, with a stryke-facing module on top. It is semantic where `stryke-gui` is physical — one calls `library.search` and gets rows; the other moves the mouse — and the two are complementary rather than competing.

The grammar is ordinary stryke, the same seen in `stryke-gui` examples: `use App`, `val` bindings, `Module::fn(...)` calls, `$handle->method(...)`, `p` to print, and `"${x}"` interpolation.

```
#!/usr/bin/env stryke
use App

# in-process: the script runs inside the app (palette step / hook)
val $me = App::here()
val @hits = @ { $me->call("library.search", %{ q => "graphene" }) }
p "found ${\ scalar @hits} items"

# out-of-process: drive running apps by name, from the shell
val $cite = App::open("zcite")
val $req = App::open("zreq")

# cross-app: every DOI in the zcite selection fires a request in zreq
for val $it (@ { $cite->get("selection") }) {
  $req->call("request.send", %{ url => "https://api.crossref.org/works/${ $it->{doi} }"
  ~ })
}

# subscribe: when zcite adds an item, log it
$cite->on("itemAdded", fn ($item) { p "added: ${ $item->{title} }" })
```

The whole surface, in seven calls

Call	Returns	Notes
<code>App::here()</code>	handle	the app the script runs inside (in-process only)
<code>App::open(\$name)</code>	handle	dial a running app's socket; dies if absent
<code>App::list()</code>	list	bus names of running apps
<code>\$h->verbs()</code>	manifest	typed verbs / state / events, for introspection
<code>\$h->call(\$verb, %args)</code>	value	invoke a verb, await the result, decode it
<code>\$h->get(\$state)</code>	value	read a state query

Call	Returns	Notes
<code>\$h->on(\$event, \$fn)</code>	subscription	run a callback per emitted event

Seven calls is the entire vocabulary a script needs to enumerate, drive, read, and watch any app in the suite. Everything else is the app's own verbs.

The handle is the unit of connection

Each call in that table returns or consumes a *handle*, and understanding what a handle is explains why the vocabulary is exactly seven and not more. A handle is a live binding to one app: in-process it wraps the host callback into the local surface; out-of-process it wraps an open socket to one app's `.sock` file. `App::here` and `App::open` are the two ways to mint one — the first for the app the script runs inside, the second for a named app across the socket — and `App::list` is the way to discover which names `App::open` would succeed against right now. Everything else is a *method on a handle*, because everything else is a thing you do *to a particular app*: enumerate its verbs, call one, read its state, watch its events. The vocabulary is small because the model is small — a script holds one handle per app it drives, and it drives each app through that app's handle and no other. There is no global `call(app, verb)` free function, and the absence is deliberate: routing every call through a handle keeps the target explicit at every call site, so a script that drives four apps reads as four clearly separated conversations rather than one undifferentiated stream of `calls` distinguished only by their first argument.

A fuller worked handle

The opening example showed the four operations one line each; a real handle is used across many lines, and the shape of that use is worth seeing whole. Here a single `zreq` handle enumerates itself, reads state, sends a request, and subscribes, all against the one open connection:

```
#!/usr/bin/env stryke
use App

val $req = App::open("zreq")

# 1. enumerate - discover the request verbs this build actually exposes
for val $v (@{ $req->verbs()->{verbs} }) {
    if $v->{id} =~ /^request\.\/ { p "  ${ $v->{id} }" }
}

# 2. read - what environment is active right now
val $env = $req->get("activeEnvironment")
p "active environment: ${ $env->{name} }"

# 3. call - send one request and bind its response
val $res = $req->call("request.send", %{ url => "https://example.com/health" })
p "status ${ $res->{status} } in ${ $res->{ms} }ms"

# 4. subscribe - log every future send this app performs
$req->on("requestSent", fn ($r) { p "sent: ${ $r->{method} } ${ $r->{url} }" })
```

One handle, four operations, one app. Every larger orchestration in Chapter 16 is this pattern multiplied across handles: one handle per app, each handle driven through the same four methods, the script the only place any two apps' values ever meet.

Chapter 6 – The Transport

Two modes, one API

The script never learns which transport it is using, but there are two.

In-process is for a script running inside the app — a palette `stryke` step or a hook. `App::here()` binds directly to the local `ZGui.automation` surface through the host callback. There is no socket and no serialization round trip beyond the JSON the bridge already speaks; it is the lowest-latency path.

Out-of-process is for a standalone file in the shell driving a separately running app. Each app process, on boot, opens a Unix domain socket:

```
$XDG_RUNTIME_DIR/zgui/<app>.sock      # Linux
$TMPDIR/zgui/<app>.sock                # macOS (XDG_RUNTIME_DIR usually unset)
```

`App::open("zcite")` dials `zgui/zcite.sock`. The host is a shared Rust helper — the `zgui-bridge` crate — so the per-app cost is a single line in `main: serve(app_name, surface)`. The precedent is again concrete: `zterminal` already speaks a raw wire protocol straight to a socket, the `tmux imsg` protocol, with no subprocess in between. The bus applies the same discipline.

A newline of JSON

The wire is deliberately small: newline-delimited JSON frames, request/response with an attached subscription stream.

```
-> {"t":"call","id":1,"verb":"library.search","args":{"q":"graphene"}}
<- {"t":"reply","id":1,"ok":true,"value":[ {...}, {...} ]}

-> {"t":"get","id":2,"state":"selection"}
<- {"t":"reply","id":2,"ok":true,"value":[ {...} ]}

-> {"t":"verbs","id":3}
<- {"t":"reply","id":3,"ok":true,"value":{"verbs":[...],"state":[...],"events":[...] }}

-> {"t":"sub","id":4,"event":"itemAdded"}
<- {"t":"reply","id":4,"ok":true}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"..."}}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"..."}}

<- {"t":"reply","id":N,"ok":false,"error":"no such verb: foo.bar"}
```

The `id` field correlates each reply to its request; the `sub` field correlates each pushed event to its subscription. Crucially, the in-process transport speaks the *same frames* over the host callback rather than a socket, which means the `App` module has exactly one codec to maintain. The two transports differ only in what carries the bytes.

The life of a socket

A socket file is a small piece of shared state on the filesystem, and shared state on the filesystem has a life cycle that has to be managed or it rots. The `zgui-bridge` host manages it in the obvious, boring way, which is the correct way. On boot, `serve(app_name, surface)` computes the path — `zgui/<app>.sock` under the runtime directory — creates the `zgui/` parent if it is missing, unlinks any stale file already sitting at that path, binds a fresh `UnixListener`, and sets the mode to `0600` so that only the owning user can dial it. The unlink-before-bind step is what makes a crashed app recoverable: a process that dies without cleaning up leaves a dead socket file behind, and the next launch simply removes it and takes the name.

This produces one honest race, and the design names rather than hides it. Two copies of the same app launched at once will both try to own `zcite.sock`; the second to bind wins the file, and the first is left serving a socket no dialer will find because the name now points elsewhere. The stance is that this is correct: the suite's apps are single-instance by intent — one running `zcite`, one command palette, one front door — so “the last one to bind owns the name” is not a bug to paper over but the single-instance rule expressing itself at the socket layer. A future launch-on-demand, if it comes, is where this would be tightened; version one lets the last writer win.

On clean shutdown the host unlinks its own socket, so a script's `App::list()` — which enumerates the live `*.sock` files — reflects reality within the window between a crash and the next launch, and exactly reflects it otherwise. `App::open` against a name whose file is gone fails at `connect`, not at some later call, so a script learns immediately that its target is down rather than sending a frame into a socket that no one is reading.

Framing: why newlines, and what they cost

The wire is newline-delimited JSON, and the choice of a newline as the frame boundary is the kind of small decision that pays for itself every day it is not noticed. A newline delimiter means the reader is a `BufReader::read_line` loop and the writer appends `\n` after each serialized value — no length prefix to compute and keep in sync with the body, no chunked framing to reassemble, no binary header to version. The cost is exactly one constraint: a frame's JSON must contain no literal newline, which is free because JSON string escaping already turns an embedded newline into `\n` inside the encoded string, so a valid JSON value serialized without pretty-printing is always a single physical line. The bus leans on that guarantee rather than fighting it. It also means a human can watch the wire with `socat - UNIX-CONNECT: $TMPDIR/zgui/zcite.sock` and read the traffic directly, one frame per line, which is the debugging affordance Chapter 23 builds on. A binary protocol would be marginally smaller and considerably less legible; for a single-user local bus, legible wins.

One listener, many connections

A single `.sock` file is one *listener*, but a listener accepts many *connections*, and the distinction matters for how several scripts share one app. When two `.stk` files both `App::open("zcite")` at once, each `connect` produces its own accepted stream inside the app; the app serves them concurrently, each with its own `id` space, its own set of subscriptions, and its own reply slots. There is no shared per-dialer state to collide because the state that matters — the correlation table, the subscription list — is per-connection. Two scripts subscribing to `itemAdded` each get their own delivery; one script's `unsub` or disconnect never disturbs the other's stream. The single-instance rule of the previous section governs which *process* owns the name; within that one process, the number of concurrent dialers is bounded only by the app's willingness to serve them, and the shared `zgui-bridge` host serves each on its own task. So `App::list()` reporting one `zcite` does not mean one script may drive it; it means one process answers, to as many scripts as ask.

The path, resolved once

The socket path deserves one more sentence of precision because scripts sometimes need to reason about it directly. The host computes the directory as `$XDG_RUNTIME_DIR/zgui` when `XDG_RUNTIME_DIR` is set — the Linux case — and `$TMPDIR/zgui` otherwise, which is the macOS case where `XDG_RUNTIME_DIR` is usually unset. The filename is always `<app>.sock`, where `<app>` is the app's bus name — the same name `App::open` takes and `App::list` returns. This is why `App::list()` can be implemented as a directory scan: it reads the `zgui/` directory, takes every entry ending in `.sock`, and strips the suffix to recover the bus name. The directory is the registry; there is no separate discovery service to keep in sync with reality, because the presence of a bound, connectable socket file *is* the registration, and its absence *is* the deregistration.

Chapter 7 – The Hybrid Handler

What each app actually ships

The design in the earlier chapters is the contract; the shipped reality is a file named `bus.rs` in each app's Tauri backend, and it is worth reading because it resolves a tension the design implies but does not spell out. An app is not one surface. It is a Rust engine — the `*-core` crate the shell already calls — plus a frontend `appshell` with its own registered verbs. A remote `call` might target either. The handler is therefore **hybrid**, and its own header comment says so:

Engine commands — the verbs listed in the core's `commands::COMMANDS` — route straight to the same `*-core` engine the UI uses, in Rust, with no round trip through the webview. A membership test decides: `is_engine_cmd(cmd)` is simply `core::commands::COMMANDS.contains(&cmd)`. Everything else — the `appshell.*` verbs, an embedded core's verbs, anything registered through `ZGui.automation` — is forwarded to the webview: the backend evaluates a dispatch shim, `window.__zguiBridgeDispatch(id, kind, name, args)`, which runs the registered verb and reports the result back through a `zgui_bridge_reply` command that fulfills a per-request channel.

The effect is that the *entire* app shell is scriptable, not merely the engine. Fast verbs stay fast — an engine call never leaves Rust — while the full frontend surface remains reachable through the one forwarding path. The socket host itself is the shared `zgui-bridge` crate: `serve`, a `Bridge`, and a `Handler` the app implements. One socket per running process, exactly as the ownership rule requires.

Discovery versus dispatch

There is a deliberate asymmetry between what an app *lists* and what it *accepts*. `verbs()` returns the enumerable surface — the engine's `COMMANDS` plus the registered frontend verbs — so a script can discover what exists. But `call` accepts any command the handler can route, even one discovery does not advertise. `zwire` is the clearest case: its host lists a `SURFACE_VERBS` array for discovery, but the comment on that array notes that `call` accepts any `browser.<verb>`, because the Chromium HUD's own executor is the thing that runs it. Discovery is a courtesy to the script author; dispatch is governed by the allow-list the handler enforces. The two need not be identical, and where they differ, the book says so.

The routing decision, in one branch

The whole hybrid handler reduces to a single branch taken once per incoming call, and it is short enough to state exactly. The handler receives a verb name; it asks `is_engine_cmd(cmd)`, which is `core::commands::COMMANDS.contains(&cmd)`; and it forks. If the verb is an engine command, the handler decodes the arguments and calls the `*-core` engine directly, in Rust, composing the reply from the engine's return value — no webview, no JavaScript, no second process. If it is not, the verb belongs to the frontend, and the handler evaluates the dispatch shim in the webview and waits on the per-request reply channel described in Chapter 4. Two arms, one predicate. The predicate is a set membership test against a list the core already maintains for its own UI, so the handler adds no new registry of its own: the same `COMMANDS` list that tells the shell which commands the engine answers tells the bus which calls skip the webview. The handler is thin precisely because it delegates the hard question — “does the engine implement this?” — to the one authority that already knows, the core's own command list.

Namespacing an embedded core

The ownership rule of Chapter 2 says an embedded core contributes its verbs *into the host app's surface* rather than opening a socket of its own, and the hybrid handler is where that rule becomes code. An app that embeds a second engine — a mail client that embeds a calendar core, say — registers the embedded core's verbs under a distinct namespace prefix so that `calendar.event.add` and the mail engine's own `message.send` never collide, and both answer on the one socket the host app owns. The handler's routing does not change: an embedded core's verbs are either in some `COMMANDS` list it can test against, in which case they route straight to that engine in Rust, or they are frontend-registered, in which case they take the webview path like any other frontend verb. The single front door holds no matter how many engines sit behind it, because the door does not care how many rooms the house has — it cares only whether the verb a caller named is one this process, by any of its parts, knows how to answer.

Chapter 8 – The Surface, App by App

The census

The bus is no longer a proposal; it is a surface with a size. Counting the verbs each app exposes — its engine `COMMANDS` list, its host's verb array, or (for the coreless trading terminal) the entire `⌘K` palette catalog its webview registers as bus verbs — plus the shared `appShell` verbs every app inherits, the live surface is **3,073 actions across eleven apps** with an enumerable verb list, plus fifteen shared shell verbs. The catalog `GUI_SCRIPT_ACTIONS.md` is generated from each app's source, so the figure moves with the code rather than aging in prose.

App	Verbs	The surface, in one line
traderview	1726	every <code>view:</code> tile and <code>action:</code> shortcut — a charting / trading terminal whose whole <code>⌘K</code> palette is scriptable
zpdf	309	render, edit, annotate, forms, OCR, redact — an Acrobat-scale PDF engine
zcite	178	library, collections, citations, PDF, sync — a reference manager
zemail	174	accounts, folders, messages, PGP and S/MIME, search
zwire	161	tabs, windows, tab groups, downloads, reading list, power — a browser
zftp	123	FTP, SFTP, WebDAV, S3 and cloud stores, transfers, sync
zreq	121	requests, collections, auth, codegen, gRPC and WebSocket
ztunnel	97	OpenVPN and WireGuard configuration and control
zoffice	96	writer, calc, impress, over ODF and OOXML
zthrottle	54	system monitoring, process and network throttling
zgo	19	launcher system commands — sleep, lock, empty trash, and the rest
appShell	15	terminal, files, hooks, palette, theme, CRT, neon — on every app

The shared fifteen

Because every app is a shell over `zgui-core`, every app inherits the same `appshell.*` verbs without declaring them: the terminal (`appshell.terminal.open`, `.close`), the file browser (`appshell.files.open`,

`.close`), the settings and shortcuts panels, the command palette, and the visual toggles for the light and dark themes, the CRT scanline effect, and the neon glow. A script that knows only these fifteen can already open a terminal, flip a theme, or summon the palette in any app in the suite — before it learns a single app-specific verb.

Shape, not just size

The verbs are namespaced, and the namespace is the app's own vocabulary made legible. `zreq`'s surface reads as a map of Postman: `collection.*`, `environment.*`, `request.*`, `codegen.*`, `graphql.*`, `grpc.*`. `zcite`'s reads as Zotero: `library.*`, `collection.*`, `item.*`, `citation.*`. Reading a namespace is reading the product's domain model. That is what makes the surface scriptable in the human sense as well as the machine sense: the verb you want is named the thing you would call it.

Not every app keeps its verbs in a Rust list. `traderview` has no `-core` engine at all — its `bus.rs` forwards everything to the webview's `ZGui.automation` surface — yet it is the *largest* app on the bus, because its front-end registers the whole ⌘K palette catalog as verbs: every `view`: tile and `action`: shortcut, the ephemeral recents and favourites deliberately filtered out. That surface is enumerable from source (the tile table and the shortcut list), so the catalog counts it honestly. Four other coreless apps — `zphoto`, `zstation`, `ztranslator`, and `zcontainer` — carry the bus but register no comparable catalog; they expose the shared `appShell` verbs plus their Tauri backend commands, and the catalog enumerates only the shell surface for them, saying so rather than inventing a count. Honesty about the boundary of what is enumerable is part of the catalog's contract.

zcite, read as a surface

Take the reference manager as the first close read, because its surface maps cleanly onto a working librarian's day. The `library.*` namespace is the entry point — search, the entire holdings, the recently added — and it is the namespace the opening example of the book called into. `collection.*` is the shelving system: `collection.add` and `collection.rename` manage the folders, `collection.add_item` and `collection.remove_item` move items between them, and the read-only `collection.tree`, `collection.list`, and `collection.diff` report their structure. `item.*` is the item lifecycle — `item.add`, `item.add_attachment`, and the rest — and it is the namespace whose returns matter most, because `item.add` hands back the item it created, the fact Chapter 16's reading-list recipe turns on. Around that core sit the interchange namespaces: `bib.*` renders citations and bibliographies (`bib.bibliography`, `bib.citation`, `bib.csl`, `bib.styles`), while `export.*` and `import.*` are the widest namespaces of all, each carrying a couched knowledge of a format — `export.bibtex`, `export.ris`, `export.csl_json`, `export.marcxml`, `export.zotero_rdf` outbound, and the matching `import.crossref_json`, `import.pubmed_xml`, `import.datacite_json` inbound. A script that reads `library.*` to find work and `export.*` to render it has automated the two ends of the manager without touching a window.

zreq, read as a surface

The API client's surface reads as a map of the request lifecycle. `request.*` is the verb-dense center: `request.send` fires one and returns the response, `request.add`, `request.update`, `request.get`, and `request.remove` manage saved requests, and `request.fuzz` drives a request repeatedly under varied inputs. `collection.*` groups requests the way `zcite`'s groups items, and `environment.*` — `environment.activate`, `environment.add`, `environment.list`, `environment.remove` — is the variable-binding layer that makes one saved request run against staging or production by swapping the active environment rather than editing the request. The protocol namespaces widen the client past plain HTTP: `graphql.*` introspects and parses schemas (`graphql.introspection_query`, `graphql.schema.parse`), `grpc.call` invokes a gRPC method, and `codegen.generate` turns a saved request into client code in a chosen language. `history.*` closes the loop — `history.list`, `history.get`, `history.replay` — so a

script can re-run past traffic without reconstructing it. The surface is Postman rendered as verbs, and the verb you want is named the thing you would call it.

zoffice, read as a surface

The office engine splits its surface by document kind, and the split is legible in the namespaces. `calc.*` is the largest and the most script-friendly, because a spreadsheet is the most naturally programmable of the document types: `calc.open`, `calc.eval` and `calc.evaluate` for formula evaluation, `calc.edit_cell`, `calc.find` and `calc.replace`, `calc.sort`, `calc.named_ranges`, `calc.pivot_tables`, `calc.charts`, `calc.validations`, `calc.conditional_formats`, and the render/export family `calc.pdf`, `calc.csv`, `calc.html`, `calc.markdown`, `calc.render`. `writer.*` is the prose engine and `impress.*` the presentation engine (`impress.charts_render`, `impress.graphic_objects`, `impress.find`), while `draw.*` handles vector documents (`draw.connectors`, `draw.svg`, `draw.render`). The invoice recipe of Chapter 16 leans on this surface: it renders a billing range out of `calc` and hands the resulting file to the PDF engine, one export verb feeding one open verb across the socket.

zemail, zftp, and ztunnel in brief

Three more surfaces round out the picture without a full walk each. The mail client's namespaces track a mail app's nouns: `account.*` (`account.add`, `account.autoconfig`, `account.list`), `folder.*` (`folder.create`, `folder.inbox_load`, `folder.digest`), `message.*` for composition and sending, `contact.*` (`contact.import_vcard`, `contact.find_duplicates`, `contact.merge`), `filter.*` (`filter.run`, `filter.to_sieve`), and the protocol-level `imap.*` (`imap.folders`, `imap.idle`, `imap.build_search`), alongside the PGP and S/MIME security surface the census line names. The transfer client's surface is a catalog of transfer protocols: `transfer.*` (`transfer.add`, `transfer.cancel`, `transfer.resume_check`, `transfer.multipart_plan`), the wire-level `sftp.*`, `ftp.*`, and `s3.*` (`s3.presign`, `s3.list_objects`), the remote-filesystem `fs.*` (`fs.list`, `fs.mkdir`, `fs.chmod`, `fs.rename`), and `sync.*` (`sync.compare`, `sync.plan`, `sync.resolve`). The VPN client is control and configuration: `vpn.*` (`vpn.connect`, `vpn.disconnect`, `vpn.status`, `vpn.reconnect_schedule`, `vpn.killswitch_plan`), the WireGuard `wg.*` (`wg.genkey`, `wg.pubkey`, `wg.handshake_model`), the OpenVPN `openvpn.*` (`openvpn.push_reply`, `openvpn.inline_blocks`), and `dns.*` (`dns.leak_check`, `dns.split_horizon`). The tunnel recipe of Chapter 16 gates sensitive work on `vpn.status` reporting connected — the read path the old palette never had.

The coreless apps and the honest gap

The census closes on the apps whose surface the catalog deliberately under-counts, and the honesty there is worth restating as a property rather than an apology. `traderview` is the extreme in one direction: no `-core` engine at all, yet the largest surface on the bus, because its whole `⌘K` palette — every `view:` tile and `action:` shortcut — is registered as bus verbs, and the tile table and shortcut list are enumerable from source so the catalog counts them exactly. `zphoto`, `zstation`, `ztranslator`, and `zcontainer` are the extreme in the other direction: they carry the bus and answer on a socket, but they register no comparable frontend catalog, so what the generator can enumerate for them is the shared `appshell` verbs plus their Tauri backend commands, and the catalog reports that surface and no more. The number for those apps is small not because the apps are small but because the *enumerable* surface is, and the catalog's contract is to report what it can prove, never to estimate what it cannot. That discipline is why every count in Chapter 8 can be trusted: it is a floor the generator measured, not a figure a writer hoped for.

Chapter 9 — Three Front-Ends, One Module

Layer 5 is the promise that there is no per-surface logic. Three different places run `stryke`; all three get `App` in scope and use the same module.

The `palette stryke step` used to pass only a context argument. Extended, it binds `App::here()` to the current app, so a palette command stops being a single `event` fire and becomes a real orchestration — read the selection, call a verb, branch on the result — while the step-chain grammar (`url | js | scheme | event | stryke | bash`) is otherwise unchanged. The `stryke` step simply got more powerful.

The `hooks runtime` gets the identical binding, so an app's hooks can react to emitted events and call verbs in response — the app scripting itself as things happen inside it.

The `standalone .stk` file runs from the shell and uses `App::open(name)` over the socket. This is the cross-app bus: one file, one terminal, several running apps driven together. The same module serves all three because the transport difference is hidden beneath it — the palette step and the shell file call the identical `App` methods, and only the layer below knows one used a callback and the other a socket.

The palette step, before and after

The clearest way to feel the change is to put the old palette step next to the new one. Before the bus, a palette command that wanted to react to a search had no way to; the best it could do was fire one `event` step and stop, learning nothing. After the bus, the same command becomes a real branch, because the `stryke` step now has `App::here()` bound to the current app:

```
# a palette command's stryke step - runs inside the app that owns the palette
val $me = App::here()
val @hits = @{ $me->call("library.search", %{ q => $ctx->{query} }) }
if scalar @hits == 0 {
    $me->call("appshell.palette.toast", %{ text => "no matches" })
} else {
    $me->call("collection.add", %{ name => "search: ${ $ctx->{query} }" })
}
```

The step still receives its context argument, `$ctx`, exactly as it always did — that part of the contract did not change — but now the same step can read the app, branch on the result, and call a second verb. The palette did not gain a special palette API; it gained the one binding that puts the whole bus in the step's scope, and everything the standalone `.stk` file can do, the palette step can now do too, against `here()` instead of `open()`.

The hook, reacting to the app

The hooks runtime is the same binding aimed at a different trigger. A palette step runs when the user invokes it; a hook runs when the app emits an event the hook subscribes to. The identical `App::here()` is in scope, so a hook can read state and call verbs in response to the app scripting itself as things happen inside it:

```

# a hook fired on the app's own "itemAdded" event
val $me = App::here()
val $item = $ctx->{payload} # the emitted item
if $item->{type} == "journalArticle" && !$item->{doi} {
    # a new article with no DOI - try to resolve one from its title
    val @found = @{ $me->call("library.search", %{ q => $item->{title} }) }
    if scalar @found > 0 { p "possible match: ${ @found->[0]->{doi} }" }
}

```

Three surfaces, one module, one binding. The palette step reacts to the user, the hook reacts to the app, and the `.stk` file reacts to nothing — it drives from the outside. All three write the same `App` calls, and none of them contains a line of code that knows which surface it is running on. That is the whole content of Layer 5: not three integrations, but one module placed in three scopes.

Chapter 10 – Security and the Allow-List

The bus is powerful, so its security model is stated in three flat rules rather than left implicit.

First, **the socket is local and private**. It lives in the per-user runtime directory at mode 0600. There is no network listener, ever. The bus is a single-user, single-machine facility by construction, not by configuration — there is no setting that opens it to the network because there is no network code to enable.

Second, **the target must be running**. An out-of-process `call` dials a socket that exists only while the app is up; dialing a dead app fails and the script dies. Version one has no launch-on-demand, so a script cannot silently spawn apps as a side effect of automating them.

Third, **verbs are an allow-list**. Only what an app registered through `ZGui.automation.register` is reachable. There is deliberately no generic “evaluate this JavaScript in the app” verb, because such a verb would dissolve the typed surface into arbitrary code and make the allow-list meaningless. The escape hatches that do run arbitrary code — the `js` and `bash` palette steps — stay exactly where they are: explicit, user-authored steps in a local palette, never remotely callable verbs. The line between “a script the user wrote and ran” and “a verb a remote caller can invoke” is the line the security model protects.

The trust boundary, drawn precisely

It helps to state exactly where the trust boundary runs, because the three rules above are its consequences rather than its definition. The boundary is the socket file’s owner. Everything on the inside — the user’s own scripts, the user’s palette steps, the user’s hooks — is trusted with the full surface, because it is all code the one user wrote or chose to run under their own account. Everything on the outside — another user on the same machine, a process running as a different uid, anything across a network — is trusted with nothing, because mode 0600 means the socket file cannot even be opened by them. There is no middle tier, no partially-trusted caller, no capability token to scope down a dialer, because the model has exactly two states: you are the user, and you have everything, or you are not, and you have no socket to dial. The simplicity is the security property. A model with a middle tier has to get the middle tier right; a model with no middle tier has only the file-permission bit to get right, and the operating system gets that bit right for a living.

Why there is no “eval” verb

The refusal to ship a generic “run this code in the app” verb deserves its own treatment, because it is the single decision that keeps the allow-list meaningful. A verb that evaluated arbitrary JavaScript, or ran an arbitrary shell command, or read an arbitrary file, would be one entry in the allow-list that made every other entry irrelevant: a caller with that one verb could do anything the app’s process could do, and the careful enumeration of `library.search`, `item.add`, and the rest would be decoration over a hole. So the surface has no such verb, anywhere, by policy. The apps that legitimately need to run arbitrary code — and some do — expose it only through the palette’s `js` and `bash` steps, which are not bus verbs at all: they are steps in a command the user typed into their own palette, on their own machine, and they are never reachable through `call`. The distinction is exact. `App:open("zcite") ->call("bash", ...)` does not exist and cannot be made to exist, because `bash` is a palette step kind, not a registered verb, and only registered verbs are

callable. The escape hatch stays an escape hatch — a thing the user reaches for deliberately in a local editor — and never becomes a remote capability.

What the allow-list is, mechanically

“Allow-list” is a precise claim about the dispatch path, not a policy statement, so it is worth grounding in the handler. When a `call` frame arrives, the handler resolves the verb name against exactly two sources: the engine’s `COMMANDS` list and the set of verbs registered through `ZGui.automation`. A name in neither set is not dispatched to some default handler that guesses; it is rejected with `ok:false` and the message `no such verb: <name>`, and no app code runs. This is what makes the surface an allow-list rather than a deny-list: the default answer to an unrecognized verb is *no*, and a verb becomes callable only by being explicitly added to one of the two known sets. The one deliberate widening, described in Chapter 7, is *zwire*’s host accepting any `browser.<verb>` because its Chromium HUD is the executor — but even that is an allow-list with a wildcard *namespace*, not an open door: a frame for `foo.bar` against *zwire* is still rejected, because it is not under the one prefix the handler was told to forward. The wildcard is scoped, declared, and documented; it is the exception that shows the rule is real.

The threat the model does not address, named

An honest security chapter names what it does not defend, and this one leaves one threat explicitly out of scope: a malicious script the user runs themselves. The bus gives a script the full surface of every running app, so a hostile `.stk` file the user is tricked into executing can do real damage — send mail, delete files over `fs.delete`, bring a tunnel down. The bus does not defend against this and does not try, because it cannot: a script the user chose to run is, by the model’s own definition, inside the trust boundary, indistinguishable from a script the user wrote. The defense against a hostile script is the same defense as against any hostile program the user runs — do not run programs you do not trust — and it lives at the level of the user’s judgment, not the bus’s dispatch. Naming this is part of the model’s honesty: the bus secures the *boundary*, not the user’s choices inside it.

Chapter 11 – Novelty – An Honest Prior-Art Analysis

A combination, not a single invention

This chapter is written in the house style of [INVENTIONS.md](#): claim the smallest true thing, name the prior art, and record confidence honestly. The bus is a **combination** first, not a single new capability. Embedding a language in an application, scripting across applications, and shipping a vendor-authored automation language each predate this work, separately and by decades. The claim is the *conjunction*, under constraints — and the honest form of that claim is “none found, low confidence,” a packaging novelty, not a proven categorical first.

The four nearest prior arts each match part of the shape and fail a load-bearing leg:

Prior art	Matches	Fails on
AppleScript / OSA (1993) — 20+ scriptable first-party apps	app count, a vendor language	macOS-only; apps are Apple-Event targets and the interpreter is external, not embedded in the app
VBA — hosted in-process in Word, Excel, and the rest LibreOffice with Basic and UNO	in-process embedding, a vendor language, cross-app via COM cross-platform, embedded, scriptable across apps	not cross-platform; single domain (office) single domain; interpreted Basic with no JIT; a component model, not shared embeddable cores; not solo-authored
KDE with Kross and D-Bus	domain diversity, cross-app	Kross bridges Python, Ruby, and JavaScript rather than being a language; no owner-authored language, no JIT, IPC not shared cores

What survives the intersection

Take the intersection of all the legs and no prior art holds it: a domain-diverse suite — a PDF editor, an email client, a VPN, a browser, an office suite, and more — that is cross-platform, solo-authored, driven by the author’s own from-scratch Cranelift-JIT language, embedded in-process across shared embeddable engine cores. AppleScript is macOS-only and external; VBA is Windows-first and single-domain; LibreOffice is single-domain with no JIT; KDE has no owner language. The combination is the claim, and it is recorded as a combination — low confidence, non-exhaustive search — precisely so that the claim survives scrutiny.

The **speed leg** is the part that is not a packaging argument. `stryke` is a Cranelift-JIT VM; an in-process call is a host callback, not a fork; an out-of-process call is a Unix socket, not HTTP. There is no subprocess per call — the same discipline as `stryke-gui` holding one persistent input handle rather than spawning a tool per keystroke. Whatever else is or is not first here, the bus is built to be fast in a way that shelling out to per-platform scripting runtimes structurally cannot be.

The modern comparison, and why it also misses

The prior-art table reaches back to the nineties because the *shape* the bus occupies is old, but a fair reading has to address the modern tools too, since a reader will reach for them first. Browser automation frameworks drive one class of application through one wire protocol, but they drive an external target and speak no vendor language of their own — they are a client library, not an embedded interpreter, and they cover exactly one app. General RPC frameworks give typed cross-process calls with generated stubs, which is genuinely close on the transport leg, but they are a plumbing layer a developer builds a system *on top of*, not a scripting surface a user drives apps *through*; there is no user-facing language and no notion of a running app's live state to read. Shell-level automation tools chain commands, but a command-line tool is not an application's internal surface — it is whatever the tool's author chose to expose on the command line, which for a GUI app is usually nothing. Each modern tool matches the leg the old ones missed and misses a leg the old ones had. The bus's claim survives the modern intersection for the same reason it survives the historical one: the conjunction of an owner-authored JIT language, embedded in-process across a domain-diverse suite of shared cores, with one uniform local transport, is not the thing any single existing tool set out to be.

The confidence, stated as a discipline

The chapter ends where the house style demands: on the confidence, stated plainly and low. “None found” is a claim about a search, and the search was not exhaustive, so the honest label is “none found, low confidence” — a packaging novelty asserted as a combination, not a categorical first proven against the whole history of software. This is not modesty for its own sake; it is the only claim that survives an adversarial reading. A stronger claim — “the first ever” — invites a single counterexample to falsify it, and the space of scripting systems is too large and too poorly indexed for anyone to have checked it all. The combination claim is different: it does not assert that no one has ever built anything like this, only that the specific conjunction of legs, held all at once, is one this author has not found elsewhere and believes to be rare. That claim a counterexample refines rather than destroys, which is exactly the property a claim published in a book about a shipped thing should have.

Chapter 12 — Rollout and Open Questions

Shared once, then per app

The rollout separates the work that happens once from the work that repeats. The shared pieces are four: the `zgui-core automation.js` surface registry with its JS bridge dispatch and `emit`; the `zgui-bridge` Rust crate with the Unix-socket host, the frame codec, and the request router; the `stryke-app` package in `strykelang` with its C-ABI exports and `App` module; and the `run_stryke_hook` change in each app backend that binds `App::here()` into the script's environment.

The per-app work is deliberately thin, because the architecture front-loaded it. An app declares its surface with one `ZGui.automation.register` call — and most of those verbs already existed as palette actions, now merely typed — and serves its socket with one `zgui_bridge::serve` line in `main`. Then it is verified twice: a `.stk` script drives it in-process through a palette step, and out-of-process from the shell. The pilots were chosen for the richest existing surfaces, so the loop could be proven before the fan-out; the census in Chapter 8 is what the fan-out produced.

The questions left open

A design honest enough to publish is honest about what it has not settled. Five questions remain live. Whether `App::open` holds its socket for the script's life or redials per call — the proposal holds and reconnects on a broken pipe. How strict parameter typing is at the JSON boundary — the proposal validates against the manifest and dies with the verb's signature on a mismatch. Whether `on` callbacks run on a stryke event loop or drain a queue the script pumps — the proposal drains at an explicit pump point, to stay inside stryke's execution model. Whether the socket host is best as a shared crate or a per-app module, given the vendorable-and-durable constraint every dependency in this stack must satisfy. And whether launch-on-demand belongs in a later version, so a script can open an app that is not yet running.

None of these blocks the surface that already exists. More than three thousand verbs answer today. The open questions are about making the edges of the bus as considered as its center — which is the right kind of question to still be asking about a thing that is already built.

The pilot loop, in detail

The rollout's "verified twice" deserves unpacking, because the two verifications are not redundant — they exercise the two transports, and a surface can pass one and fail the other. The in-process verification runs a `.stk` snippet as a palette step: it binds `App::here()`, calls a verb, reads a state query, and prints the result into the palette's own output, confirming that the surface registered, that the bridge's webview path resolves the verb, and that a value comes back into the VM. The out-of-process verification runs the same logical script as a standalone file from the shell, using `App::open(name)` instead of `here()`, which additionally confirms that the socket bound, that the frame codec round-trips, and that `App::list()` sees the app. Passing in-process but failing out-of-process points at the socket host — the `serve` line missing, the path wrong, a permissions problem. Passing out-of-process but failing in-process points at the palette binding. Two transports, two failure

signatures, and running both on every pilot is what let the fan-out proceed on the confidence that a green surface was green on both paths, not just the one the author happened to test.

Why the pilots were the richest apps

Choosing the pilots for their *richest existing surfaces* looks backward — surely the simplest app is the safest first move — but the logic is sound and worth stating. A rich surface exercises the most of the shared machinery on the first try: many verbs with many parameter shapes stress the argument validator, state queries stress the `get` path, events stress the subscription stream, and a large `COMMANDS` list stresses the hybrid handler's engine-versus-frontend split. A trivial app with three verbs would have proven almost nothing about the shared pieces, so a bug in the codec or the pump would have surfaced only later, during fan-out, when it was more expensive to trace across a dozen apps at once. Proving the shared substrate against the hardest surface first means the fan-out inherits a substrate already tested against everything it will face — the eleventh app to join found nothing left to break that the first, hardest one had not already found.

Chapter 13 – The Value Model at the Edge

What crosses the wire

JSON is the only thing on the wire, so the value model on the wire is JSON's: null, boolean, number, string, array, object, and nothing else. stryke's model is richer — scalars, arrays written with `@`, maps written with `%`, live handles, closures — and so every argument going out and every result coming back is a projection between the two. The projection is total in one direction and partial in the other, and the partiality is where the interesting decisions live.

Outbound, from script to app, the projection is total. An argument map like `%{ q => "graphene", limit => 20 }` becomes a JSON object; a stryke array becomes a JSON array; scalars become the matching JSON scalars. Two stryke values have no JSON image and so cannot be arguments, and this is a feature rather than a gap. A closure cannot cross, because there is nothing on the far side to run it against and no way to serialize a body of JIT-compiled code. A handle cannot cross either, because a handle is a live connection and not a value — a point Chapter 5 states and this chapter makes load-bearing below. The one place a callback appears in the API, `on`, never serializes it: the closure stays resident in the VM and only the subscription `id` travels, so events are delivered by pumping frames through a function that never left home.

Inbound, from app to script, the projection has one lossy seam and is otherwise clean. A JSON array decodes to a stryke array a script walks with `for val $x (@{ ... })`; a JSON object decodes to a map indexed as `$it->{doi}`; JSON `null` decodes to stryke `nil`, which is exactly the value a script tests for when a state query has no answer. The seam is numbers. JSON has a single numeric type; stryke distinguishes integer from float. The codec restores an integer when the decoded value is integral and a float otherwise, which is precisely right for the overwhelming majority of verbs — a row count, an id, a page number are integers and arrive as integers — and merely conventional for the rare verb whose contract is an integral-valued float. A script that needs to be certain coerces at the point of use; the codec's rule is the one that surprises the fewest callers.

Why a handle is not a value

The rule that a handle cannot be an argument is worth dwelling on, because it shapes the whole topology of a multi-app run. A handle names a transport plus a target: it is, concretely, an open socket to `zcite` or a bound callback into the local surface. Serializing that and shipping it to another process would mean shipping a socket, which means nothing on the far end. So handles are VM-local by construction. `App::open` mints one, the handle methods consume it, and it dies when the script does.

The consequence is architectural, not merely mechanical. A script that wants `zreq` to act on data living in `zcite` does not — cannot — hand `zreq` a handle to `zcite` and step back. It reads from `zcite` itself and calls into `zreq` itself, staying the hub through which every value passes. The topology is therefore always a star, with the script at the center and each app a spoke, and no app ever holds a live connection to another app. That is the same shape the security model wants for its own reasons — an app is never a client of another app, only ever a server to the user's own scripts — and it falls out for free from the single decision that a handle is not a value. Two chapters, one constraint, no coordination needed between them.

The number seam, worked

The one lossy seam — JSON's single numeric type against stryke's integer/float split — is worth working through a concrete case, because it is the projection decision a careful script author will eventually hit. Consider `library.search` returning items, each carrying a `year` field. On the wire the year is JSON `1998`, a bare number with no type tag. The codec's rule is: decode an integral value to a stryke integer and a non-integral value to a stryke float. So `1998` becomes the integer `1998`, and `$item->{year} + 1` is integer arithmetic, which is what a script author expects. The rule only surprises in the rare inverted case: a verb whose contract is a float that happens to be integral — a price of exactly `20.0`, say, arriving on the wire as `20` — decodes to the integer `20`, and a script that then formats it as currency has to know to coerce. The codec chooses the rule that surprises the fewest callers rather than the rule that is theoretically lossless, because there is no lossless choice: JSON threw the type away before the codec ever saw the value. A script that needs certainty coerces at the point of use — `$price * 1.0` forces a float — and the codec's convention handles the overwhelming common case for free.

The three values that cannot cross, together

Chapter's earlier sections named the non-serializable values one at a time; seen together they form a single rule with one reason. A closure cannot cross because there is no far side to run it against. A handle cannot cross because it is a live connection, not a value. A stryke value carrying either transitively — a map with a closure inside it — cannot cross whole, because the projection to JSON has no image for the interior. The unifying reason is that JSON is a *data* format and these are *behavior and connection*, not data. The bus does not paper over the gap with a fake serialization that would deserialize to something broken on the far side; it refuses the value at the boundary, loudly, so the script author learns at the call site that they tried to send something that is not sendable. This is the same posture as the failure model of the next chapter: fail where the fault is, not three statements later where a broken value finally gets dereferenced.

Chapter 14 — Failure Is a Reply

The two ways a call can end

Every `call`, `get`, and `verbs` request ends in exactly one reply frame, and that frame's `ok` field is the fork in the road. `ok:true` carries a `value`; `ok:false` carries an `error` string. The `App` module turns the two into two different things inside the VM, and the difference is deliberate. A true reply decodes its `value` and returns it to the calling expression. A false reply raises, and an unhandled raise dies the script with the error string as its message. So `no such verb: foo.bar` is not a sentinel a script must remember to test for — it is a death, loud and immediate, the same kind of death as a nil dereference or a failed `App::open`.

This is a design choice with a defensible reason. A mistyped verb name in an automation script is a bug in the script, not a branch the script meant to take. Returning nil for an unknown verb would let the mistake propagate silently until some later line dereferenced the nil and blamed the wrong statement. Dying at the call site, with the verb's own name in the message, puts the error where the fault is. Scripts that genuinely want to probe for a verb's existence read `verbs()` and test membership; they do not fish by calling and catching. The fast path — call and use — stays free of defensive ceremony because the slow path fails honestly.

The failures the wire cannot phrase

Some failures never become a reply frame at all, because the channel that would carry the reply is itself gone. An out-of-process target that exits in the middle of a call breaks the pipe; there is no `ok:false` because there is no peer left to compose it. The `App` module surfaces a broken pipe as its own kind of death, naming the app that vanished, rather than letting the script block forever on a socket no one will ever write to — a hang being the single worst outcome for an automation script, worse than any error, because it produces no signal at all.

This is exactly the design surface behind the Chapter 12 open question about holding versus redialing a socket. The proposal holds the connection for the handle's life and reconnects once on a broken pipe: a target that restarts cleanly between two iterations of a loop does not kill a script that would otherwise have marched on. The retry is single and bounded, because a socket that refuses twice in a row is a target that is genuinely down, and a script driving a down app should learn so and stop, not spin.

Timeouts

There is a third failure that neither an `ok:false` nor a broken pipe describes: a verb that accepts the call, never errors, and never resolves. In-process, a `run` that awaits a promise that never settles would hang the calling script as completely as a dead socket. The bridge therefore treats every `call` as a bounded wait rather than an open-ended one. The request carries an `id`; the reply must arrive under a deadline; a deadline that passes with no reply becomes the same kind of death as a broken pipe, again naming the verb. This is the one moment the synchronous-looking call shows the asynchronous machine beneath it. To the script, `App::call` is a function that returns a value or dies — nothing more. Underneath, the bridge is running a timer against a correlation id, and the tidy synchronous surface is the timer's success case made to look like the only case.

The full taxonomy of a failed call

Gathered into one place, a `call` can fail in exactly four ways, and a script author who holds the taxonomy in mind writes calmer automations. First, the *rejected* call: the verb ran or was refused and the app composed an `ok:false` reply — an unknown verb, a missing required argument, an app-level error the verb chose to raise. This is the common case, it names the fault, and it raises in the VM. Second, the *broken* call: the peer vanished mid-flight, the pipe broke, and no reply frame is possible; the `App` module manufactures a death naming the app that disappeared. Third, the *timed-out* call: the peer is alive but the verb never resolved under the deadline; the bridge's timer fires and manufactures a death naming the verb. Fourth, the *undialed* call: the target was never reachable, `App::open` failed at `connect`, and the script died before any frame was sent. Four failures, and the design's through-line across all of them is that every one is a *death at the call site*, never a sentinel value that propagates. The reason is uniform: an automation script that hangs, or that marches on with a nil where a value should be, produces no signal or the wrong signal, and both are worse than a loud death with the fault named. The bus spends its complexity to guarantee that a call either returns the value or stops the script — nothing quieter.

Catching, for the scripts that mean to

A death is the default, not a mandate, and a script that genuinely wants to survive a failed call can. stryke's own error handling wraps a call whose failure is a branch the author actually meant to take rather than a bug — probing an app that may or may not answer a given verb, tolerating one flaky request in a batch, treating a down tunnel as a condition to report rather than a reason to abort:

```
val $res = try { $req->call("request.send", %{ url => $u }) }  
           catch ($e) { p "skipped ${ $u }": ${ $e }"; nil }  
if $res { record($res) }
```

The point is that catching is *opt-in*. The fast path — call and use the value — carries no defensive ceremony, because the default is a death that stops the script at the fault. Only a script that has a real reason to continue past a failure reaches for the `try`, and when it does, the failure it catches is the same four-way taxonomy above, delivered as a stryke error whose message is the reply's `error` string or the manufactured broken-pipe or timeout text. The author who wants a branch writes one; the author who wants correctness by default writes nothing and gets it.

Chapter 15 – Order, Correlation, and the Pump

Ids are a conversation, not a clock

The `id` on a request is a correlation token and not a timestamp, and reading it as a clock is the mistake the transport is built to survive. A Unix socket is full-duplex, so a script is free — in a future pipelined mode, or simply because the kernel buffers — to have several requests outstanding at once, and their replies may come back in any order the app happens to finish them. The `App` module matches each reply to the waiting request by `id`; it assumes nothing about arrival order.

In today's straight-line scripts this machinery is invisible, because the synchronous `App:call` blocks on its own `id` before the next statement runs, so there is only ever one request in flight and reordering has nothing to reorder. The correlation matters for the events that share the channel, and for any later mode that lets a script fire a batch and gather the answers. It is built now, cheaply, so that the wire never has to be redesigned to add pipelining later — the `id` was always going to be the thing that made concurrency safe, so it is the thing on every frame from version one.

Events do not interrupt

Replies and events travel the same socket, and the design flatly refuses to let an event preempt a running script. `on` registers a callback, but the callback does not fire the instant its frame arrives off the wire. Firing on arrival would mean arbitrary stryke code running in the middle of an unrelated expression — a value half-computed, a map half-built, the stack in a state the author never wrote — and stryke's execution model does not permit that, for the same reason no sane language lets a signal handler resume inside an arithmetic operation.

So events queue. They accumulate in a buffer attached to the subscription and they drain at an explicit pump point, and this is the concrete resolution of the Chapter 12 open question about whether `on` runs on an event loop or against a pumped queue. It is a pumped queue. A callback runs only when the script pumps, which means the script author always knows precisely the set of places a callback can run: the pump points, and nowhere else. Control flow is never violated from outside. The cost is symmetrical and honest — a script that subscribes and then never pumps will never see one event — and it is the right trade, because “my callbacks run where I say” is worth far more to an automation author than “my callbacks run the instant the frame lands.”

The pump in practice

The pump is ordinary stryke, not a special form. A long-running watcher subscribes, then loops, pumping and doing its own work between drains:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val @seen = @{}

$cite->on("itemAdded", fn ($item) {
    push @seen, $item->{doi}
    p "added: ${ $item->{title} }"
})

# drain the subscription queue on a cadence the script controls
loop {
    App::pump()                # run any queued callbacks now
    if scalar @seen >= 10 { break } # stop after ten additions
    App::sleep(250)
}
p "collected ${\ scalar @seen} new DOIs"
```

The subscription is live for the whole loop, but a callback body runs only at the `App::pump()` line — never while `push` is mid-flight, never inside the `if`. The script reads top to bottom exactly as written, and the events fold into it at one named seam. That is the entire contract, and it is enough.

What the queue guarantees, and what it does not

The pumped queue makes two promises and withholds a third, and knowing which is which keeps a watcher correct. It promises *order*: events from one subscription drain in the order they arrived off the wire, so a script that watches a sequence of additions sees them in sequence, not shuffled. It promises *completeness up to the pump*: every event that landed before a `pump` call runs during that call, none skipped, so a script that pumps regularly misses nothing that arrived. What it withholds is *timeliness*: an event that lands the instant after a `pump` returns waits, unseen, until the next `pump`, and a script that stops pumping stops seeing events entirely. This is the deliberate trade named earlier — “my callbacks run where I say” bought at the price of “my callbacks do not run the instant the frame lands” — and the two withheld and promised properties are the same coin’s two faces. A watcher that wants tight latency pumps in a tight loop; a watcher that wants to do heavy work between drains pumps less often and accepts the lag. The queue never drops and never reorders; it only ever *delays*, and the delay is under the script’s own hand.

Backpressure at the queue

A subscription’s queue is a buffer, and a buffer that fills is a design question the pump model answers rather than dodges. If an app emits faster than a script pumps — a burst of a hundred `itemAdded` events while the script sleeps between drains — the events accumulate in the subscription’s buffer, and the next `pump` runs all hundred callbacks in a row. The buffer is bounded, so a script that subscribes to a firehose and never pumps does not grow memory without limit; past the bound, the oldest queued events are dropped and the drop is observable, so a watcher that cares can notice it fell behind rather than silently missing events it never knew existed. The design’s stance is that a script driving a busy app is responsible for pumping often enough to keep up, exactly as a script reading a busy socket is responsible for reading it, and the bounded buffer with an observable drop is the honest middle between an unbounded buffer that hides the problem and a zero buffer that loses every event between pumps. Chapter 24 returns to this as the general backpressure story; here it is enough that the queue has a ceiling and says so when it hits it.

Chapter 16 – Orchestrations

The preceding chapters justify the bus one mechanism at a time. This one spends the mechanism. Each recipe below is a whole script that exists only because the bus does — several running apps, one file, one terminal — and each is read for the same thing: the line where a value leaves one app and enters another, which before the bus had no way to be written.

From a selection to a batch of requests

The smallest orchestration is two apps and one loop, and it is the example the [App](#) module chapter opened with, now read for what each line costs. The script reads the current selection out of the reference manager and, for every item, fires a crossref lookup in the API client:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $req  = App::open("zreq")

for val $it (@{ $cite->get("selection") }) {
  val $doi = $it->{doi}
  if $doi {
    $req->call("request.send",
              %{ url => "https://api.crossref.org/works/${ $doi }" })
  }
}
```

`$cite->get("selection")` is a state read that returns a list of maps across the socket; `$it->{doi}` reaches into one of those maps; `$req->call("request.send", ...)` crosses to a second, unrelated app. Before the bus, the selection lived behind a window the script could not read and the request lived behind a palette button that returned nothing. The bus does not add a feature to either app. It makes the value that was always there in `zcite` reachable, and the verb that was always there in `zreq` callable, from the same six lines.

Harvesting the browser's reading list

A richer recipe crosses the browser and the reference manager in the other direction. The browser, [zwire](#), keeps a reading list; the script pulls it, and for every entry that looks like a paper it adds the item to the library and files it in a named collection:


```
#!/usr/bin/env stryke
use App

val $wire = App::open("zwire")
val $cite = App::open("zcite")

val @list = @{ $wire->get("readingList") }
p "reading list holds ${\ scalar @list} entries"

for val $entry (@{ @list }) {
  val $url = $entry->{url}
  val $doi = doi_from_url($url)          # a plain stryke helper, no app involved
  if $doi {
    val $item = $cite->call("item.add", %{ doi => $doi })
    $cite->call("collection.addItem",
      %{ collection => "from-reading-list", item => $item->{id} })
    p "filed: ${ $item->{title} }"
  }
}
```

Two things are worth naming. First, `item.add` returns the item it created — the whole reason the bus exists is that this return value lands in `$item` and the very next line uses `$item->{id}`; a fire-and-forget action could not have done this at all. Second, `doi_from_url` is not an app verb; it is ordinary stryke running in the VM between two calls. The bus does not force every step through an app. It lets app calls and plain computation sit in the same straight-line program, which is what makes a real workflow expressible as one readable file.

The invoice run: three apps, one pipeline

The strongest case for the bus is the one no single app could ever answer, because it lives in the space between three. A calc sheet holds a billing range; the office suite renders it, the PDF engine finishes it, and the mail client sends it:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")
val $pdf    = App::open("zpdf")
val $mail   = App::open("zemail")

# 1. export the current calc sheet's billing range to a PDF on disk
val $out = $office->call("calc.exportRange",
  %{ sheet => "Invoices", range => "A1:F40", format => "pdf" })

# 2. stamp it in the PDF engine - flatten, watermark, protect
val $doc = $pdf->call("document.open", %{ path => $out->{path} })
$pdf->call("document.watermark", %{ doc => $doc->{id}, text => "PAID" })
$pdf->call("document.flatten",    %{ doc => $doc->{id} })
val $final = $pdf->call("document.save", %{ doc => $doc->{id} })

# 3. attach and send it
$mail->call("message.send", %{
  to      => "billing@example.com",
  subject => "Invoice - ${ $out->{period} }",
  attach  => @{ $final->{path} },
})
p "sent ${ $final->{path} }"
```

Read the seams. `$out->{path}` leaves the office suite and becomes the argument that opens a document in

the PDF engine. `$doc->{id}` threads three PDF verbs together. `$final->{path}` leaves the PDF engine and becomes a mail attachment. No file was copied by hand, no app was clicked, no format was guessed — each app did the one thing it is best at and handed its result to the next, and the handoff is a stryke variable. This is the shape the opening sentence of the book promised: one language driving every app by name, and getting the answer back.

Gating on the tunnel

The last recipe uses the bus for control rather than data. Before running any verb that touches a sensitive host, the script brings a VPN up through `ztunnel`, waits for the state query to confirm the link, does its work, and tears the tunnel down — so the sensitive calls provably run only while the tunnel is up:

```
#!/usr/bin/env stryke
use App

val $vpn = App::open("ztunnel")

$vpn->call("connection.up", %{ profile => "work" })

# poll the state query until the link reports connected
val $tries = 0
loop {
  if $vpn->get("status")->{state} == "connected" { break }
  if ($tries += 1) > 20 { die "tunnel never came up" }
  App::sleep(500)
}

# ... sensitive verbs against internal hosts run here, tunnel guaranteed up ...

$vpn->call("connection.down", %{ profile => "work" })
p "tunnel down, run complete"
```

The `get("status")` read in the loop is the bus doing what the old palette never could: reading an app's live state and branching on it. A fire-and-forget `connection.up` followed by a blind `sleep` would only *hope* the tunnel was ready; reading `status` until it says `connected` *knows*. The difference between hoping and knowing is exactly the return path the bus adds, and here it is load-bearing for correctness, not merely convenience.

Watching one app to drive another

The recipes so far are straight-line; this one is event-driven, and it is the first to spend the pump of Chapter 15 in anger. The script watches the browser's downloads and, whenever a PDF finishes, opens it in the reference manager as a new attachment — a running bridge between two apps that reacts rather than polls:

```
#!/usr/bin/env stryke
use App

val $wire = App::open("zwire")
val $cite = App::open("zcite")

$wire->on("downloadComplete", fn ($dl) {
  if $dl->{path} =~ /\.pdf$/ {
    val $item = $cite->call("item.add", %{ file => $dl->{path} })
    p "filed download: ${ $item->{title} }"
  }
})

loop { App::pump(); App::sleep(500) }    # drain the download stream forever
```

Two seams matter. The `on` callback fires only inside `App::pump()`, so the `item.add` call into `zcite` never interrupts a half-built expression — the guarantee of Chapter 15 doing real work. And the callback, running in the VM, freely calls a *second* app: the browser's event drove a reference-manager verb, and neither app knew the other existed. The script is the only place the download and the library ever met.

Fan-out with per-item failure tolerance

The invoice recipe assumed every step succeeded; a real batch does not. This recipe sends a request per row of a calc sheet and tolerates the ones that fail, collecting a result table rather than dying on the first bad row — the `try` of Chapter 14 spent across a fan-out:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")
val $req     = App::open("zreq")

val @rows = @{ $office->call("calc.cells", %{ sheet => "Endpoints", range => "A2:A200" }) }
val @ok    = @{}
val @bad   = @{}

for val $row (@{ @rows }) {
  val $url = $row->{value}
  if !$url { next }
  val $res = try { $req->call("request.send", %{ url => $url }) }
              catch ($e) { push @bad, %{ url => $url, why => "${ $e }" }; nil }
  if $res && $res->{status} < 400 { push @ok, $url }
  elsif $res { push @bad, %{ url => $url, why => "status ${ $res->{status} }" } }
}

p "ok: ${\ scalar @ok}    bad: ${\ scalar @bad}"
for val $b (@{ @bad }) { p "    ${ $b->{url} } - ${ $b->{why} }" }
```

The shape is the one every real batch wants and the old palette could not express: read a range out of one app, drive a second app once per row, distinguish the three outcomes — hard failure, HTTP failure, success — and finish with a report instead of a crash. `calc.cells` returns the range as data the script walks; `request.send` returns each response into a variable the script inspects; the `try` turns a broken call into a recorded row rather than a dead script. Every value that crosses between the two apps crosses through the script, and every failure that could have stopped the run becomes a line in the final table.

Reconciling two apps' views of the same data

The last recipe uses the bus to *compare* rather than to move or to gate. The reference manager and the transfer client both know about a set of files — the manager's attachments and the client's remote store — and the script reconciles them, reporting what exists in one place but not the other:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $ftp = App::open("zftp")

# every attachment the library knows about, by filename
val %have = %{}
for val $it (@{ $cite->call("library.search", %{ q => "" }) }) {
  for val $att (@{ $it->{attachments} }) { %have{ $att->{name} } = true }
}

# what the remote store holds
val @remote = @{ $ftp->call("fs.list", %{ path => "/backup/library" }) }

# the two-way diff, computed in the script
for val $f (@{ @remote }) {
  if !%have{ $f->{name} } { p "on server, not in library: ${ $f->{name} }" }
}
p "library holds ${\ scalar keys %have} attachments; server holds ${\ scalar @remote}
  ~ files"
```

No single app could answer “what is on the server that the library does not track,” because the question spans two apps' private views, and before the bus there was no seam where those two views could be read into one program and compared. The recipe is the plainest possible statement of what the bus is *for*: it is not that either app gained a feature, but that a script can now hold both apps' answers at once and reason across them.

Chapter 17 – The Surface as Its Own Documentation

`verbs()` is the manual

The manifest a script reads with `$h->verbs()` is not a second, hand-written document that can drift from the code — it is the same registration the app booted with, stripped of its function bodies. That identity is what lets the surface be its own documentation. A script author who does not remember whether the reference manager calls it `item.add` or `item.create` does not consult a wiki; they ask the running app:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
for val $v (@{ $cite->verbs()->{verbs} }) {
  p "${ $v->{id} }${ join(", ", @ { $v->{params} } |map| .{name}) }" -> ${ $v->{returns} }
}
}
```

The output is a signature list generated from the exact surface the app will answer to a moment later. There is no version skew possible, because there is no second source: the thing that documents the verb and the thing that runs the verb are one registration read two ways, precisely the “one declaration, two readers” property Chapter 3 built and here collects its dividend.

The generated catalog

`GUI_SCRIPT_ACTIONS.md` — the catalog the census in Chapter 8 counts — is the same idea lifted from run time to build time. Rather than a human enumerating verbs into prose that ages, a generator walks each app's source: the engine's `commands::COMMANDS` list, the host's discovery array, and the frontend's `ZGui.automation.register` surface, plus the shared `appShell` verbs every app inherits. The catalog is therefore a projection of the code, and every count in it — the 3,073 across eleven apps, the fifteen shared shell verbs — is a number the generator recomputed, not a number a writer typed. When a verb is added to an app, the catalog grows on the next generation; when one is renamed, the catalog renames. Prose that claims a count is a snapshot that starts drifting the instant it is written; a generated catalog is a measurement that is re-taken on demand. The book leans on the second and is careful, wherever a number appears, to say which one it is.

A REPL and a completion source

Two small tools fall out of the surface being enumerable, and both are worth naming because they cost almost nothing once the manifest exists. The first is a REPL: a loop that reads a verb and an argument map from the terminal, calls it, and prints the decoded reply — a scriptable app's equivalent of a database shell, built from `verbs()` for discovery and `call` for execution, with no per-app code. The second is shell completion. Because `App::list()` enumerates the running apps and `verbs()` enumerates each app's verbs, a completion function can offer the live set of app names after `App::open(` and the live set of verb ids after a handle's `->call(`,

so a script author tab-completes against exactly what is reachable at that moment rather than against a static list that may be stale. Neither tool is a new capability of the bus. Both are the surface's own reflectiveness, spent twice.

The signature is the docstring

There is one more dividend the reflective surface pays, and it closes the loop between the machine reader and the human one. Because `verbs()` yields a verb's parameter names, parameter types, and return type, the same enumeration that lets a REPL offer completions lets a documentation generator emit a signature line no human wrote and no human can let drift:

```
library.search(q: string, collection?: string, limit?: int) -> list<item>
item.add(doi: string) -> item
collection.add_item(collection: string, item: string) -> bool
```

Those lines are not typed into a wiki; they are printed from the manifest the app booted with, and they are exactly what the app will answer to. A documentation page built this way cannot describe a verb the app does not have, cannot omit a verb the app does have, and cannot show a parameter the app dropped, because it is not a description of the surface at all — it is the surface, rendered. The distinction the whole chapter turns on is that between *documenting* a thing and *projecting* it: a document is a second artifact that must be kept in sync, and projection is the same artifact shown a second way, which cannot be out of sync because there is only one of it. Every tool in this chapter — the `verbs()` self-description, the generated catalog, the REPL, the completion source, the signature line — is projection, not documentation, and that is the single property that makes each of them free and none of them able to lie.

Chapter 18 – The Long Arc

Launch-on-demand

The sharpest edge version one leaves unground is that an out-of-process `call` requires its target to already be running; dial a dead app and the script dies at `connect`. This is a deliberate floor, not an oversight — a script that could silently spawn a dozen apps as a side effect of automating them would be a surprising and heavy thing — but it is also the most-requested next move. The shape of the answer is visible from here: a small resolver that, on a failed `connect`, consults a registry mapping bus name to launchable bundle, starts it, and waits for its socket to appear before retrying the dial once. The security model constrains the design more than it complicates it: launch-on-demand must be opt-in per script and must never cross the single-user boundary, because a facility that spawns apps is exactly the facility an attacker would most like to borrow. Version one earns the right to add this later by refusing to fake it now.

The remote temptation, refused

The most natural-seeming extension is the one the design most firmly declines. A newline-JSON protocol over a Unix socket is one `bind` away from being the same protocol over a TCP socket, and a TCP socket is one config flag away from being reachable across a network. The bus will not take that step, and Chapter 10's flat rule — there is no network listener, ever — is written precisely so that the temptation has no toehold. The reasoning is that the bus's whole security posture rests on the socket being a single-user, single-machine artifact at mode `0600`; the moment a frame can arrive from another host, every verb becomes a remote procedure call and the allow-list has to defend against callers it was never designed to distrust. Cross-machine automation is a real and different problem with real and different answers — authentication, transport security, capability scoping — and folding it into the local bus by loosening one bind call would be the kind of shortcut that dissolves a clean model. If cross-machine ever ships, it ships as its own thing with its own threat model, not as this socket with a wider bind.

Typed stubs for the language

Chapter 3 drew the line at the manifest's types being a boundary check and a document rather than a shared type system, and Chapter 18 is where the richer version is named. Because `verbs()` yields a machine-readable signature for every verb — parameter names, parameter types, return type — a generator could emit stryke stub modules per app: `Zcite:library_search($q, $collection)` as a real, autocompleting, arity-checked function rather than a `call("library.search", ...)` string the VM only validates at dispatch. The stubs would be regenerated from the live surface, so they could not drift, and a script that used them would fail at compile time on a misspelled verb or a missing argument instead of at run time on a dead reply. This is the same move the generated catalog makes — code projected from the surface — aimed at the language instead of at documentation. It is deferred, not dismissed: the string-keyed `call` is the honest primitive, and the typed stubs are sugar poured over it once the primitive has proven itself across the full census.

The bus as a platform primitive

Step back far enough and the bus stops looking like a feature of a scripting language and starts looking like a property of the whole suite. Every app in the architecture book was already a thin shell over a pure-Rust core sharing one component library; the bus is the observation that such a suite is, without any further invention, a single programmable surface — that “the shell can call the engine” and “any script can call the engine” are the same sentence with a socket inserted. What the suite gains is not a dozen automation APIs but one, uniform across a PDF editor and an email client and a browser and a VPN, spoken in the same seven calls, carried by the same newline of JSON. A new app joins the platform by registering a surface and serving a socket — two lines — and inherits the whole tooling, the REPL, the completion, the catalog, the orchestration idioms, for free.

That is the arc the book has been tracing from its first sentence. One language should be able to drive every app you own, by name, and get an answer back. The bus is the small set of pieces that makes the sentence true, and the reason it could be small is that the architecture had already done the hard part: it had already turned every app into a set of functions waiting to be called. The bus only had to hand them a caller.

Chapter 19 – Writing a New App Module

The two lines, expanded

Chapter 18 closed on the claim that a new app joins the bus in two lines — register a surface, serve a socket. That claim is true, and this chapter is what those two lines look like when they are the whole of a real morning's work rather than a slogan. The exercise is concrete: take an app that ships today with no automation surface and put it on the bus, from the first `register` call to a green `.stk` script driving it from the shell. Nothing here is new mechanism; it is the mechanism of the first eighteen chapters, walked once in the order a builder actually meets it.

Step one: declare the surface

The surface lives in the frontend, owned by the core, registered once on boot. The first move is to find the app's existing action list — the `setActions` call it already makes to populate its palette picker — and rewrite it as a typed manifest. An app that today does this:

```
// before: id and label, nothing more
appShell.setActions([
  { id: "note.new",   label: "New note" },
  { id: "note.find", label: "Find notes" },
]);
```

becomes this:

```
// after: typed verbs, state, and events on the automation surface
ZGui.automation.register({
  app: "znotes",
  verbs: [
    { id: "note.new",
      label: "New note",
      params: [{ name: "title", type: "string", required: true },
               { name: "body", type: "string", required: false }],
      returns: "note",
      run: async (a) => znotes.create(a.title, a.body ?? "") },
    { id: "note.find",
      label: "Find notes",
      params: [{ name: "q", type: "string", required: true }],
      returns: "list<note>",
      run: async (a) => znotes.search(a.q) },
  ],
  state: [
    { id: "selection", returns: "list<note>", get: () => znotes.selection() },
  ],
  events: [
    { id: "noteCreated", payload: "note" },
  ],
});
```

Three things happened and none of them was expensive. The two actions kept their ids, so every palette command that already fired `note.new` still works — the id is the compatibility surface, and it did not move.

Each action grew a parameter list, a return type, and a `run` body that calls the engine method the palette action was already bound to. And two entries that did not exist before appeared: a `state` query for the current selection and an `event` the app already had a moment to emit. The picker in the palette, which read the flat list, now reads the typed surface and shows richer entries for free — the “one declaration, two readers” property of Chapter 3, collected the moment the surface is written.

Step two: emit the events

A surface that declares an event but never pushes down it is a promise unkept, so the second move is to find the one place in the engine where the thing happens and add the `emit`. For `noteCreated` that is wherever a note is actually created:

```
async function create(title, body) {
  const note = await znotes.engine.insert({ title, body });
  ZGui.automation.emit("noteCreated", note);
  return note;
}
```

One line, at the point of the event, fire-and-forget. If no script is subscribed the `emit` costs a membership check against an empty list; if one is, the bridge fans the payload out to it. The app does not know or care which, which is exactly why the `emit` can be added unconditionally and left in place forever.

Step three: serve the socket

The frontend now has a surface; the backend now needs a front door. That is the one line in `main`:

```
zgui_bridge::serve("znotes", surface_handle);
```

`serve` computes the socket path — `zgui/znotes.sock` under the runtime directory — creates the `zgui/` parent if missing, unlinks any stale file, binds a fresh `UnixListener` at mode `0600`, and begins accepting connections, each served on its own task by the shared host. The `surface_handle` is the backend’s route into the app: the hybrid `Handler` that tests each incoming verb against the engine’s `COMMANDS` and either runs it in Rust or forwards it to the webview’s dispatch shim. For an app whose verbs all live in the frontend — like this notes app — the handler forwards everything; for an app with a rich `-core`, the engine verbs stay in Rust. Either way the line in `main` is the same, because the routing lives in the shared crate, not in the app.

Step four: verify twice

The app is on the bus; now prove it, on both transports, exactly as the pilots did. In process first, as a palette `stryke` step:

```
val $me = App::here()
val $n = $me->call("note.new", %{ title => "from the palette" })
p "created note ${ $n->{id} }"
```

Then out of process, as a standalone file from the shell, which additionally proves the socket bound and `App::list` sees the app:

```
#!/usr/bin/env stryke
use App

p "running apps: ${\ join(", ", @{ App::list() }) }"      # znotes should appear
val $nn = App::open("znotes")
val $n  = $nn->call("note.new", %{ title => "from the shell" })
p "created note ${ $n->{id} }"
for val $hit (@{ $nn->call("note.find", %{ q => "shell" }) }) {
  p "  found: ${ $hit->{title} }"
}
```

If the palette step prints an id but the shell file cannot find the app, the fault is the socket line — missing, mis-pathed, or a permissions problem. If the shell file finds the app but a call is rejected with **no such verb**, the fault is the surface — a typo in an id, or a verb declared but not registered. The two-transport verification is not ceremony; it splits the failure space in half on the first run, which is the whole reason the rollout insisted on it.

What the tutorial did not require

It is worth naming what did not appear in the four steps, because the absences are the architecture paying off. No new socket protocol was designed; the app inherited the frame codec. No serialization was hand-written; the bridge projects JSON to stryke values and back. No discovery service was registered; the bound socket file *is* the registration. No per-app tooling was built; the REPL, the completion source, and the generated catalog all read the new surface with no code specific to it. The new app did not integrate *with* the bus so much as *appear on* it, because the two lines it wrote were the only two lines that could have been app-specific, and everything else was already shared. That is the difference between joining a platform and wiring up an integration, and it is the whole return on the architecture book's up-front discipline.

Chapter 20 – The Subscription and Event Model

A subscription's life, frame by frame

Chapter 4 introduced subscription as the streaming half of the one substrate and Chapter 15 governed when its callbacks run; this chapter follows a single subscription through its whole life, because the events surface has corners the earlier chapters passed over. A subscription begins with a `sub` frame and a local registration. When a script writes `$cite->on("itemAdded", $fn)`, two things happen at once: the `App` module stores the closure `$fn` in a table keyed by a fresh subscription id, entirely inside the VM, and it sends `{"t":"sub","id":4,"event":"itemAdded"}` down the wire. The app replies `{"t":"reply","id":4,"ok":true}` to confirm the registration, and from that moment the app's `emit("itemAdded", payload)` pushes an `{"t":"event","sub":4,"event":"itemAdded","payload":{...}}` frame for every occurrence. The `sub` field on each event frame is the subscription id, and it is how the `App` module finds the right closure to run: an event frame arrives, the module looks up `sub` in its table, and queues the payload for that closure's buffer. The closure never traveled; only its id did, and the id is the whole correlation.

Ending a subscription

A subscription that never ends is a leak, so the model has three ways to end one, and a careful watcher knows which it is using. The explicit end is `unsub`: the handle a `->on` call returns can be dropped or closed, which sends `{"t":"unsub","id":4}` and removes the closure from the VM table, after which the app stops pushing that stream. The implicit end is the handle's own death: when the `App::open` handle goes out of scope and its socket closes, every subscription on that connection ends at once, because the app sees the connection drop and tears down all per-connection state — the subscriptions, the reply slots, everything. The third end is the script's: when the `.stk` file finishes, the VM tears down, the socket closes, and the app cleans up on the disconnect exactly as it would for any dropped connection. The design leans on the second and third far more than the first, because a script that subscribes and then exits is the common case, and making disconnect a clean, total teardown means most scripts never write `unsub` at all — they just end, and the app notices.

Filtering at the source versus in the callback

An event stream can be narrowed in two places, and the choice has a cost the script author should make deliberately. The stream can be narrowed *in the callback*, which is what the download-watcher recipe of Chapter 16 did: it subscribed to every `downloadComplete` and tested `$dl->{path} =~ /\.pdf$/` inside the closure, letting the non-PDF events arrive and be ignored. That is simple and it keeps all the filtering logic in the script, but it pays to deliver events the script will discard. The stream can instead be narrowed *at the source*, if the app's event declaration supports a filter argument on `sub` — an app that emits a single `fileChanged` event for a whole tree can accept a path-prefix filter so a subscriber hears only its subtree. Source-side filtering delivers less over the wire but pushes the filter's definition into the app's contract; callback-side filtering delivers everything but keeps the script in sole control. Version one leans on callback-side filtering as the honest default — every event declared is delivered whole, and the script decides what it cares about — because it keeps the

event contract as small as the verb contract: a name and a payload, no per-subscription query language to design and version. Source-side filtering is the same kind of deferred sugar as the typed stubs of Chapter 18: real, useful, and left for an edition that has proven it needs it.

Replay, and why the bus does not offer it

A question every event system eventually faces is whether a late subscriber can catch up on events it missed — replay from a log. The bus deliberately does not offer this, and the refusal is of a piece with its whole posture. An event on this bus is a live notification, not a persisted record: it fires when the thing happens, it is delivered to whoever is subscribed at that moment, and it is gone. A script that needs the historical set does not subscribe and wait; it *calls a verb* that reads the current state — `library.search` for the items that exist, `history.list` for the requests already sent — because the state is the source of truth and the event is only the poke that says the state changed. Folding replay into the event stream would mean the app persisting an event log, versioning it, bounding it, and answering “since when” queries against it — a whole durable subsystem — to provide what a state-reading verb already provides better. The division of labor is clean and worth stating as a rule: *verbs and state queries answer “what is true now”; events answer “something just changed”; and neither pretends to be the other.* A subscriber that missed an event reads the state; it does not ask the stream to remember.

Chapter 21 – Testing the Bus

The surface is testable without a desktop

A recurring fear about GUI automation is that testing it requires a running desktop, a window server, and a human-shaped click. The bus escapes that fear by construction, because almost everything worth testing about it is data, not pixels. A verb's contract is its parameter validation and its return shape; the transport's contract is that a frame in produces the right frame out; the codec's contract is that a stryke value projects to JSON and back without loss it did not warn about. None of those needs a screen. The bus is testable at three levels — the surface, the transport, and the end-to-end script — and only the third wants a running app, which is why the first two carry most of the test weight.

Level one: the surface, in isolation

A registered surface is a data structure with functions in it, and its validation behavior can be exercised directly. A surface test constructs a manifest, hands it a call frame, and asserts on the reply — no socket, no webview, no app. The cases that matter are the boundary cases the bridge is responsible for: a call missing a required parameter must be rejected before `run` is entered; a call with a parameter of the wrong JSON kind must be rejected with the verb's signature in the message; an unknown verb must produce `no such verb`; a well-formed call must reach `run` with exactly the declared arguments. Each of these is an assertion on a reply frame given a request frame, and each runs in microseconds with no I/O. The `run` bodies themselves are the app's own code, tested by the app's own suite; the surface test's job is the *boundary* — that the bridge admits the callers it should and rejects the ones it should — and that boundary is pure enough to test as a table of frames in and frames out.

Level two: the transport, over a pair of sockets

The transport's job is to carry frames faithfully, and it can be tested against itself with no app at all: bind a `UnixListener`, connect to it, and run the frame codec in both directions. A transport test asserts the properties the wire promises — that a frame written is a frame read, that newline framing survives a value containing escaped newlines, that two requests with distinct ids get replies matched to the right id even when the mock server answers them out of order, that a subscription's event frames carry the right `sub`, and that a mid-call disconnect surfaces as the broken-pipe death rather than a hang. The mock server on the far end is a few lines — accept, read a frame, write a canned reply — and it lets the test drive every failure mode of Chapter 14's taxonomy deterministically: to test the timeout death, the mock accepts the call and simply never replies; to test the broken-pipe death, it accepts and then closes; to test reordering, it holds two calls and replies to the second first. The transport's hardest behaviors are its failure behaviors, and a mock server makes each of them a reproducible test rather than a race that shows up once a month in the wild.

Level three: the end-to-end script, against a running app

The top level is the one that wants a real app, and it is the smallest, because the lower two have already proven the pieces. An end-to-end test launches the app, runs a `.stk` file against it over the socket, and asserts on the script's output — the same two-transport verification the rollout used, now as a repeatable test rather than a one-time check. This level is where a whole-app regression shows: a verb renamed without updating its registration, a socket line dropped from `main`, an event that stopped firing. It is deliberately thin because it is the expensive level — it needs the app up — so it tests the *wiring*, that the app appears on `App::List` and answers a representative call, and leaves the exhaustive per-verb behavior to the surface tests below it. The pyramid is the ordinary one: many fast surface tests, fewer transport tests, a thin cap of end-to-end tests, and the reason it works is that the bus was built in layers that each have a testable boundary, so a bug can be caught at the cheapest level that is responsible for it.

Testing a script's logic without any app

There is a fourth kind of test that is easy to miss: the logic of an orchestration script, tested against a fake bus. Because a handle is reached only through the seven `App` methods, a test can substitute a fake `App` whose `call` and `get` return canned values and whose `on` feeds scripted events, and run the real orchestration against it. The reconciliation recipe of Chapter 16 — the two-way diff between the library and the server — is pure logic once its two `calls` are stubbed: feed it a fixed attachment list and a fixed remote listing, and assert it reports exactly the right difference. This is where the discipline that a handle is the only way to reach an app pays a testing dividend the security chapter did not foresee: a seam narrow enough to be a security boundary is also narrow enough to be a mock point, and the same seven methods that let the bus swap a socket for a callback let a test swap a real app for a fixture.

Chapter 22 – Versioning the Contract

What “the contract” actually is

A contract can only be versioned once it is named, so begin there. The bus's contract has three parts, and they age at three different rates. The *wire* is the frame format — the `t`, `id`, `verb`, `args`, `ok`, `value`, `error`, `sub`, `event`, `payload` fields — and it is the slowest-moving part, because every app and every script speaks it and a change breaks everyone at once. The *surface* is each app's set of verbs, state queries, and events, with their parameter and return types, and it is the fastest-moving part, because it grows every time an app adds a feature. Between them is the *module*, the seven `App` calls, which moves rarely because it is the language-facing API a hundred scripts import. Versioning the bus means versioning these three separately, because a policy that treated them as one thing would either freeze the fast-moving surface to protect the slow-moving wire or destabilize the wire every time a verb was added.

The surface grows without breaking

The surface's compatibility rule is the one a working author leans on daily, and it is the ordinary rule for any versioned API, stated for this one. *Adding* is always safe: a new verb, a new state query, a new event, or a new *optional* parameter on an existing verb breaks no script, because a script that did not use the new thing yesterday does not use it today. *Removing or renaming* is the breaking move: a verb that changes its id, a required parameter that appears where none was, a return type that drops a field a script read — each of these can break a script written against the old shape. The rule falls out of the same “id is the compatibility surface” principle the tutorial leaned on: the id is the promise, so keeping the id stable while widening what sits behind it is the whole discipline of a non-breaking change. An app that must rename a verb keeps the old id as an alias for a version, dispatching both to the same `run`, and drops the alias only after the scripts have moved — the same deprecation courtesy any long-lived API extends, made cheap here because an alias is one more entry in the registration.

The generated catalog is the changelog

The bus has a property most versioned APIs would envy: its surface is enumerable, so its changes are diffable without anyone writing them down. `GUI_SCRIPT_ACTIONS.md` is generated from the live surface, so two generations of it, diffed, *are* the changelog — every verb added shows as an added line, every rename as a delete plus an add, every dropped parameter as a changed signature. A release process that regenerates the catalog and diffs it against the last release sees exactly what changed in the contract, with no human enumerating it and therefore no human forgetting to. This is the “projection, not documentation” property of Chapter 17 spent on versioning: because the catalog is a measurement of the surface rather than a description of it, the measurement's diff is the change, and a breaking change cannot hide from a diff of a projection the way it can hide from prose a writer updated by hand and missed a line of.

The wire's forward door

The wire moves slowest, but it will move, and the frame format left itself one forward door for when it does: the frames are JSON objects keyed by field name, so a new field can be added to a frame and an old reader will

ignore it. A future pipelined mode that wanted a `priority` on a call frame, or a batched mode that wanted a `batch` field, adds the field and every existing reader — which looks up fields by name and ignores the rest — keeps working. The `t` field is the frame's discriminant and the one field that must never change meaning, because it is how a reader decides what kind of frame it holds; a genuinely new frame *kind* gets a new `t` value, which old readers reject as unknown rather than misinterpret. This is the standard tagged-union discipline, and it is why the wire can gain capabilities — pipelining, batching, source-side event filters — by addition rather than by a flag day. The contract was designed additive from version one, which is the same instinct that put the correlation `id` on every frame before anything needed it: build the extensible shape now, so the extension later is an addition and never a redesign.

Chapter 23 – Debugging and Tracing Requests

The wire is legible on purpose

When a call misbehaves, the first advantage the bus hands you is that its traffic is readable text. A newline of JSON per frame means the whole conversation between a script and an app can be watched live with a socket relay:

```
$ socat -v UNIX-CONNECT:$TMPDIR/zgui/zcite.sock -
> {"t":"call","id":1,"verb":"library.search","args":{"q":"graphene"}}
< {"t":"reply","id":1,"ok":true,"value":[{"id":42,"title":"..."}]}
```

That transcript is the ground truth. If a script reports an empty result, the transcript says whether the empty came back over the wire — an app-side issue — or whether the call never went out — a script-side issue. If a call hangs, the transcript shows whether the `call` frame was even written, and whether any `reply` came back. The legibility that Chapter 6 chose the newline format for is not an aesthetic preference; it is the single most useful debugging affordance the bus has, because it lets you see exactly what crossed the boundary rather than inferring it from behavior on either side.

Reading the three server-side logs

The app's own side of the conversation is traced too, and it goes where the whole suite's diagnostics go: the app's log, never the terminal. The bridge logs three things at the boundary, each at a level a script author can raise when hunting a problem. It logs *acceptance* — a new connection dialed the socket — which confirms `App::open` reached the app. It logs *dispatch* — a verb name arrived and routed to the engine or the frontend — which is where a `no such verb` shows its cause, because the log names the verb the handler failed to resolve. And it logs *reply* — the frame composed for a call, with its `ok` and, on failure, its error — which is the app-side twin of the `ok:false` the script received. Three log lines per call, correlated by the same `id` the wire uses, so a single request can be followed from the script's `call`, across the `socat` transcript, into the app's acceptance-dispatch-reply trace, and back to the script's returned value or death. The `id` is the thread through the logs exactly as it is the thread through the code.

The four failures, each with a signature

The failure taxonomy of Chapter 14 doubles as a debugging guide, because each of the four failures leaves a different fingerprint across the transcript and the logs. A *rejected* call shows a `call` frame out and an `ok:false` reply back, with the error naming the fault — the app answered, and the answer was no. A *broken* call shows a `call` frame out and then the connection closing with no reply — the transcript simply ends — and the app log shows the connection dropping mid-dispatch. A *timed-out* call shows a `call` frame out, no reply, and the connection *still open* — the distinguishing sign from a broken pipe — with the app log showing dispatch begun but no reply composed, which points at a `run` body that never resolved. An *undialed* call shows nothing on any socket, because `App::open` failed at `connect` before a frame was sent, and the only trace is the script's own death at the open. Learning to read which of the four fingerprints is present turns “the script is broken”

into “the verb accepted the call and never resolved, so the bug is in its `run` body” in the time it takes to glance at the transcript.

A tracing REPL

The REPL of Chapter 17 becomes a debugging instrument when it prints the wire alongside the result. A verbose mode that echoes the `call` frame it sends and the `reply` frame it receives turns the REPL into an interactive probe: type a verb and its arguments, watch the exact frame go out, watch the exact frame come back, and read the decoded value the `App` module produced from it. This is the fastest way to answer the question a misbehaving script raises — “is the app returning what I think it is?” — because it puts a human in the loop at the exact seam where the script’s assumption meets the app’s reality. The REPL costs nothing to build, per Chapter 17, and in verbose mode it costs nothing more to make it the primary tool for understanding a verb’s real behavior before a script is ever written against it.

Chapter 24 – Performance and Backpressure

Where the time goes

A performance claim is only honest if it names where the time actually goes, so account for it call by call. An in-process call — a palette step or a hook driving `App::here()` — is a host callback into the backend, a dispatch into the webview or a direct engine call, and a return; there is no socket, no serialization beyond the JSON the bridge already speaks, and no process boundary crossed. Its cost is dominated by the verb's own work, not by the bus. An out-of-process call adds exactly one Unix-socket round trip and two JSON serializations — request out, reply back — over the in-process cost. A Unix socket round trip on a loopback is measured in microseconds, not milliseconds, because it never touches a network card or a network stack; it is a memory copy between two processes mediated by the kernel. The structural claim of Chapter 11 is grounded here: there is no subprocess per call, no interpreter start-up, no HTTP framing, no TLS handshake — the costs that make per-platform scripting-runtime automation slow are absent by construction, and what remains is a socket write, a socket read, and the work the verb was asked to do.

The engine fast path

The hybrid handler's engine path is where the bus is fastest, and it is worth isolating why. When a call names an engine verb — one in the core's `COMMANDS` — the handler runs it in Rust, in the same `-core` engine the UI calls, and composes the reply without a single frame crossing into JavaScript. For an out-of-process caller that is one socket round trip and one Rust function call; for an in-process caller it is barely more than a Rust function call. The webview path — for frontend-registered verbs — adds the shim evaluation and the reply command, which is more, but it is paid only by verbs that genuinely live in the frontend. The design's performance shape is therefore that the common, hot verbs — the engine's own commands, which are most of the surface on a rich app — take the cheapest path, and only the frontend-specific verbs take the longer one. Speed is not uniform across the surface, and the design made sure the fast path is the one the dense part of the surface travels.

Backpressure on calls: the block is the brake

Backpressure on the call path is handled by the simplest possible mechanism: the synchronous `App::call` blocks until its reply, so a script cannot issue calls faster than the app answers them. There is no unbounded send queue to overflow because there is no send queue at all in today's straight-line mode — one call is outstanding at a time, and the script waits for each before issuing the next. This is backpressure by construction: the slowest component in the loop, whichever it is, sets the pace, and the faster components idle waiting for it rather than piling work on it. A future pipelined mode that let several calls be outstanding would have to add real backpressure — a bound on in-flight calls, a block when the bound is reached — but version one gets the property for free from the blocking call, and it is the right default, because an automation script almost always wants each step's result before deciding the next step anyway.

Backpressure on events: the bounded queue

The event path is where backpressure is a live design question, because events flow app-to-script on the app's schedule, not the script's, and Chapter 15 named the answer: a bounded per-subscription queue. An app emitting faster than a script pumps fills the queue; the next pump drains it; a queue that hits its bound drops its oldest entries and makes the drop observable. The stance is that a script watching a busy stream is responsible for pumping often enough to keep up, and the bounded queue is the honest failure mode when it does not — memory stays bounded, and the script can detect that it fell behind rather than silently missing events. The alternative designs are both worse: an unbounded queue trades a detectable drop for an undetectable memory leak, and a zero-length queue drops every event that lands between pumps, which is most of them. The bounded queue with an observable drop is the middle that keeps the common case correct — a script that pumps promptly loses nothing — while bounding the pathological case rather than hiding it.

The cost of the bus at rest

A last accounting: what does an app pay for carrying the bus when no script is driving it? The answer is nearly nothing, and the nearly is worth naming. At rest, the bus is a bound `UnixListener` with no connections, a registered surface sitting as an in-memory data structure, and the `emit` calls the app makes at its own event points, each a membership check against an empty subscription list. There is no polling loop, no background thread spinning, no periodic work; the listener blocks in the kernel waiting for a `connect` that may never come, and a blocked listener costs a file descriptor and nothing else. An app on the bus that no one is automating is, to a first approximation, an app not on the bus at all — which is the property that let the fan-out reach every app without anyone weighing whether a given app was “worth” the runtime cost. The cost at rest is a file descriptor and a data structure, and both are free.

Chapter 25 – A Per-App Orchestration Cookbook

How to read this cookbook

Chapter 16 justified the bus with orchestrations chosen to show a mechanism; this chapter is the working cookbook, organized by the app at the center of each recipe rather than by the mechanism it demonstrates. Each recipe is a whole `.stk` file solving a problem a person actually has, and each is grounded in verbs the census establishes. The verb ids are written in the live snake-case form the catalog reports — `collection.add_item`, `vpn.connect`, `calc.cells` — so a reader can lift a recipe and expect it to dial a real surface. Where a recipe needs a helper that is not an app verb, the helper is ordinary stryke, marked as such, to keep the line between “an app did this” and “the script did this” visible.

zcite: build a bibliography from a search

The reference manager’s most common automation is to turn a query into a formatted bibliography without touching the UI. Search, collect the hits into a named collection, and render the collection as a bibliography in a chosen style:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")

val @hits = @{ $cite->call("library.search", %{ q => "graphene oxide" }) }
$cite->call("collection.add", %{ name => "graphene-review" })
for val $it (@{ @hits }) {
  $cite->call("collection.add_item",
    %{ collection => "graphene-review", item => $it->{id} })
}
val $bib = $cite->call("bib.bibliography",
  %{ collection => "graphene-review", style => "ieee" })
p $bib->{text}
```

The recipe spends four namespaces — `library`, `collection`, `bib` — and every value that threads them is a return the old palette could not have given back: the hits from `library.search`, the item ids into `collection.add_item`, the rendered text out of `bib.bibliography`.

zreq: run a saved collection against a fresh environment

The API client’s daily automation is to run a whole saved collection against a chosen environment and summarize the results — a smoke test expressed as a script:

```
#!/usr/bin/env stryke
use App

val $req = App::open("zreq")

$req->call("environment.activate", %{ name => "staging" })
val @reqs = @{ $req->call("collection.list", %{ collection => "smoke" }) }
val $fail = 0
for val $r (@{ @reqs }) {
  val $res = try { $req->call("request.send", %{ id => $r->{id} }) }
              catch ($e) { $fail += 1; p "ERR ${ $r->{name} }: ${ $e }"; nil }
  if $res && $res->{status} >= 400 {
    $fail += 1
    p "FAIL ${ $r->{name} }: ${ $res->{status} }"
  }
}
p "smoke: ${\ scalar @reqs} requests, ${ $fail } failures"
if $fail > 0 { die "smoke suite failed" }
```

`environment.activate` swaps the variable bindings, `collection.list` enumerates the saved requests, and `request.send` runs each — the client's own features, driven in a loop the client's UI has no way to express, ending in an exit status a CI job can read.

zoffice: a calc sheet as a data source

The office engine's most useful automation is to treat a spreadsheet as the source of truth for a batch, reading a range as data and driving other apps from it. Here a sheet of email addresses drives a mail merge through the mail client:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")
val $mail    = App::open("zemail")

val @rows = @{ $office->call("calc.cells",
  %{ sheet => "Recipients", range => "A2:B200" }) }

for val $row (@{ @rows }) {
  val $name = $row->{a}
  val $email = $row->{b}
  if !$email { next }
  $mail->call("message.send", %{
    to      => $email,
    subject => "Your invitation",
    body    => "Hello ${ $name }, ...",
  })
}
p "sent ${\ scalar @rows} invitations"
```

`calc.cells` returns the range as rows of data; the script walks them and fires `message.send` per row. The spreadsheet is the loop's data and the mail client is its effect, and the only place the two meet is the four lines of stryke between them.

zftp: mirror a local tree to a remote store with a dry run first

The transfer client's automation earns its keep on repeatable syncs, and the recipe that matters is the one that plans before it acts — compute the diff, show it, and only then transfer:

```
#!/usr/bin/env stryke
use App

val $ftp = App::open("zftp")

# plan: what would change, without changing anything
val @plan = @{ $ftp->call("sync.plan",
    %{ local => "~/site", remote => "/var/www/site" }) }

p "planned changes:"
for val $step (@{ @plan }) { p "  ${ $step->{op} } ${ $step->{path} }" }

# act: enqueue each planned transfer and watch them complete
for val $step (@{ @plan }) {
    if $step->{op} == "upload" {
        $ftp->call("transfer.add",
            %{ src => $step->{local}, dst => $step->{remote} })
    }
}
p "enqueued ${\ scalar @plan} transfers"
```

`sync.plan` returns the diff as data the script prints and then consumes, and `transfer.add` enqueues each upload. The plan-then-act shape is only expressible because `sync.plan` hands its answer back rather than acting on it — the return path turning a blind sync into a reviewable one.

ztunnel and zthrottle: a guarded, throttled backup

The last recipe composes three concerns — a tunnel, a throttle, and a transfer — into one guarded operation: bring up the VPN, throttle the backup process so it does not saturate the link, run the transfer, then restore both:


```

#!/usr/bin/env stryke
use App

val $vpn = App::open("ztunnel")
val $thr = App::open("zthrottle")
val $ftp = App::open("zftp")

# 1. bring the tunnel up and confirm it
$vpn->call("vpn.connect", %{ profile => "backup" })
loop {
  if $vpn->call("vpn.status", %{ })->{state} == "connected" { break }
  App::sleep(500)
}

# 2. cap the transfer process's network so the link stays usable
val @procs = @ { $thr->call("proc.snapshot", %{ }) }
for val $p (@ { @procs }) {
  if $p->{name} =~ /zftp/ {
    $thr->call("proc.throttle", %{ pid => $p->{pid}, net_kbps => 4000 })
  }
}

# 3. run the backup, guaranteed tunneled and throttled
$ftp->call("transfer.add", %{ src => "~/archive", dst => "/backup/archive" })

# 4. restore
$vpn->call("vpn.disconnect", %{ profile => "backup" })
p "backup complete, tunnel down"

```

Three apps, three concerns, one file. `vpn.status` gates the work on a confirmed link; `proc.snapshot` and `proc.throttle` cap the transfer's bandwidth by pid; `transfer.add` does the copy. No single app could have expressed "back up this tree, but only while the tunnel is up and only at a bandwidth that leaves the link usable," because the sentence spans three apps' surfaces — and the script is the one place all three answer at once.

Chapter 26 – The Verb Reference

What this reference is, and how it relates to the catalog

This chapter is the printed companion to the generated catalog `GUI_SCRIPT_ACTIONS.md`. The catalog is the exhaustive, machine-generated census — read from each running app's live surface and re-taken on demand — and it is always the authority on the exact set of verbs a given build exposes. This reference is the human's map of that territory: a namespace-by-namespace walk of each app's surface, naming the namespaces and a representative set of the verbs in each, so a reader can learn the *shape* of an app's surface and know what to call `verbs()` for. Two conventions hold throughout. First, verb ids are written in the live snake-case form the catalog reports; where a chapter example earlier in the book used a lightly-cleaned illustrative name, the live id is the one here. Second, this reference names representative verbs per namespace, not every verb — the exhaustive list is the catalog's job, and the census in Chapter 8 gives the exact per- app counts. Read this to learn an app's vocabulary; read the catalog to enumerate it.

The shared shell: `appshell.*`

Every app inherits the shared shell verbs, because every app is a shell over `zgui-core`. A script that knows only these can open a terminal, browse files, flip a theme, or summon the palette in any app in the suite before learning a single app-specific verb.

Verb	Effect
<code>appshell.terminal.open / .close</code>	show or hide the embedded terminal
<code>appshell.files.open / .close</code>	show or hide the file browser
<code>appshell.palette</code>	summon the command palette
<code>appshell.settings</code>	open the settings panel
<code>appshell.shortcuts</code>	open the keyboard-shortcuts panel
<code>appshell.theme.light / .dark</code>	switch the color theme
<code>appshell.crt.on / .off / .toggle</code>	the CRT scanline effect
<code>appshell.neon</code>	the neon-glow visual toggle

zcite — the reference manager

Namespace	Representative verbs	Purpose
<code>library.*</code>	<code>library.search</code>	search and enumerate holdings

Namespace	Representative verbs	Purpose
<code>collection.*</code>	<code>collection.add</code> , <code>collection.rename</code> , <code>collection.add_item</code> , <code>collection.remove_item</code> , <code>collection.tree</code> , <code>collection.list</code> , <code>collection.diff</code> , <code>collection.merge</code>	folders and their membership
<code>item.*</code>	<code>item.add</code> , <code>item.add_attachment</code>	item lifecycle; <code>item.add</code> returns the created item
<code>bib.*</code>	<code>bib.bibliography</code> , <code>bib.citation</code> , <code>bib.csl</code> , <code>bib.styles</code> , <code>bib.cite_key</code> , <code>bib.sort_key</code> , <code>bib.disambiguate</code>	rendering citations and bibliographies
<code>export.*</code>	<code>export.bibtex</code> , <code>export.biblatex</code> , <code>export.ris</code> , <code>export.csl_json</code> , <code>export.marxml</code> , <code>export.mods</code> , <code>export.endnote_xml</code> , <code>export.zotero_rdf</code> , <code>export.markdown</code> , <code>export.csv</code>	outbound format conversion
<code>import.*</code>	<code>import.bibtex</code> , <code>import.crossref_json</code> , <code>import.datacite_json</code> , <code>import.pubmed_xml</code> , <code>import.ris</code> , <code>import.endnote_xml</code> , <code>import.zotero_rdf</code>	inbound format conversion
<code>attachment.*</code>	<code>attachment.index</code> , <code>attachment.snapshot</code> , <code>attachment.store_file</code> , <code>attachment.sniff</code>	attachment handling

zreq – the API client

Namespace	Representative verbs	Purpose
<code>request.*</code>	<code>request.send</code> , <code>request.add</code> , <code>request.update</code> , <code>request.get</code> , <code>request.remove</code> , <code>request.fuzz</code>	the request lifecycle; <code>request.send</code> returns the response
<code>collection.*</code>	<code>collection.list</code>	grouping saved requests

Namespace	Representative verbs	Purpose
<code>environment.*</code>	<code>environment.activate</code> , <code>environment.add</code> , <code>environment.list</code> , <code>environment.remove</code>	variable-binding environments
<code>graphql.*</code>	<code>graphql.introspection_query</code> , <code>graphql.parse</code> , <code>graphql.schema.parse</code>	GraphQL introspection and parsing
<code>grpc.*</code>	<code>grpc.call</code>	gRPC method invocation
<code>codegen.*</code>	<code>codegen.generate</code>	turn a request into client code
<code>history.*</code>	<code>history.list</code> , <code>history.get</code> , <code>history.replay</code> , <code>history.clear</code>	past traffic

zoffice – the office engine

Namespace	Representative verbs	Purpose
<code>calc.*</code>	<code>calc.open</code> , <code>calc.cells</code> , <code>calc.edit_cell</code> , <code>calc.eval</code> , <code>calc.find</code> , <code>calc.replace</code> , <code>calc.sort</code> , <code>calc.named_ranges</code> , <code>calc.pivot_tables</code> , <code>calc.charts</code> , <code>calc.validations</code> , <code>calc.conditional_formats</code> , <code>calc.csv</code> , <code>calc.html</code> , <code>calc.markdown</code> , <code>calc.pdf</code> , <code>calc.render</code>	the spreadsheet engine
<code>writer.*</code>	<code>writer.open</code> , <code>writer.find</code> , <code>writer.replace</code> , <code>writer.render</code>	the prose engine
<code>impress.*</code>	<code>impress.charts_render</code> , <code>impress.charts_detail</code> , <code>impress.graphic_objects</code> , <code>impress.find</code>	the presentation engine
<code>draw.*</code>	<code>draw.connectors</code> , <code>draw.svg</code> , <code>draw.render</code> , <code>draw.find</code> , <code>draw.markdown</code>	vector documents

zemail – the mail client

Namespace	Representative verbs	Purpose
<code>account.*</code>	<code>account.add</code> , <code>account.autoconfig</code> , <code>account.list</code> , <code>account.update</code> , <code>account.remove</code>	account setup
<code>folder.*</code>	<code>folder.create</code> , <code>folder.delete</code> , <code>folder.rename</code> , <code>folder.list</code> , <code>folder.inbox_load</code> , <code>folder.digest</code>	mailbox folders
<code>message.*</code>	<code>message.send</code>	composition and sending
<code>contact.*</code>	<code>contact.add</code> , <code>contact.list</code> , <code>contact.merge</code> , <code>contact.import_vcard</code> , <code>contact.export_vcard</code> , <code>contact.find_duplicates</code>	the address book
<code>filter.*</code>	<code>filter.add</code> , <code>filter.apply</code> , <code>filter.run</code> , <code>filter.to_sieve</code> , <code>filter.match</code>	message filters, exportable to Sieve
<code>imap.*</code>	<code>imap.folders</code> , <code>imap.idle</code> , <code>imap.build_search</code> , <code>imap.parse_envelope</code>	protocol-level IMAP

The census line for `zemail` also names the PGP and S/MIME security surface; those verbs sign, encrypt, and verify messages, and a script drives them the same way it drives any other verb — the reference notes their presence without enumerating them here, deferring the exhaustive set to the catalog.

zwire — the browser

`zwire` is the app whose discovery and dispatch differ, per Chapter 7: its host lists a `SURFACE_VERBS` array for discovery but accepts any `browser.<verb>`, because the Chromium HUD's own executor runs it. The namespaces below are the shape of that surface.

Namespace	Representative area	Purpose
<code>browser.tab.*</code>	tab open, close, list, select	tab management
<code>browser.window.*</code>	window new, close, list	window management
<code>browser.tabgroup.*</code>	group create, add, list	tab groups
<code>browser.download.*</code>	download list, and the <code>downloadComplete</code> event	downloads
<code>browser.readingList.*</code>	reading-list read and add (the <code>readingList</code> state query)	reading list
<code>browser.power.*</code>	power and lifecycle controls	power

zftp – the transfer client

Namespace	Representative verbs	Purpose
<code>transfer.*</code>	<code>transfer.add</code> , <code>transfer.cancel</code> , <code>transfer.resume_check</code> , <code>transfer.list</code> , <code>transfer.clear</code> , <code>transfer.estimate</code> , <code>transfer.segments</code> , <code>transfer.multipart_plan</code>	the transfer queue
<code>sftp.*</code>	<code>sftp.build_init</code> , <code>sftp.parse_attrs</code> , <code>sftp.negotiate</code>	the SFTP wire
<code>ftp.*</code>	<code>ftp.parse_pasv</code> , <code>ftp.build_port</code> , <code>ftp.parse_mlsx</code>	the FTP wire
<code>s3.*</code>	<code>s3.list_objects</code> , <code>s3.presign</code> , <code>s3.sign</code> , <code>s3.complete_multipart</code>	S3 and cloud stores
<code>fs.*</code>	<code>fs.list</code> , <code>fs.mkdir</code> , <code>fs.delete</code> , <code>fs.rename</code> , <code>fs.chmod</code> , <code>fs.peek</code>	the remote filesystem
<code>sync.*</code>	<code>sync.compare</code> , <code>sync.plan</code> , <code>sync.resolve</code> , <code>sync.symlink_policy</code>	mirroring and reconciliation
<code>bookmark.*</code>	<code>bookmark.add</code> , <code>bookmark.list</code> , <code>bookmark.import</code>	saved connections

ztunnel – the VPN client

Namespace	Representative verbs	Purpose
<code>vpn.*</code>	<code>vpn.connect</code> , <code>vpn.disconnect</code> , <code>vpn.status</code> , <code>vpn.reconnect_schedule</code> , <code>vpn.killswitch_plan</code> , <code>vpn.autoconnect</code> , <code>vpn.logs</code> , <code>vpn.connections</code>	connection control and state
<code>wg.*</code>	<code>wg.genkey</code> , <code>wg.pubkey</code> , <code>wg.handshake_model</code> , <code>wg.allowed_ips</code>	WireGuard
<code>openvpn.*</code>	<code>openvpn.push_reply</code> , <code>openvpn.inline_blocks</code> , <code>openvpn.static_key_parse</code>	OpenVPN

Namespace	Representative verbs	Purpose
<code>dns.*</code>	<code>dns.leak_check</code> , <code>dns.split_horizon</code> , <code>dns.query_wire</code>	DNS handling
<code>profile.*</code>	<code>profile.save</code> , <code>profile.get</code> , <code>profile.export</code> , <code>profile.import</code> , <code>profile.encrypt</code> , <code>profile.decrypt</code>	profile management
<code>split.*</code>	<code>split.evaluate</code> , <code>split.route_plan</code> , <code>split.evaluate_app</code>	split-tunnel routing

zthrottle – system monitor and throttle

Namespace	Representative verbs	Purpose
<code>proc.*</code>	<code>proc.snapshot</code> , <code>proc.tree</code> , <code>proc.kill</code> , <code>proc.detail</code> , <code>proc.diff</code> , <code>proc.files</code> , <code>proc.history</code>	process inspection and control
<code>bench.*</code>	<code>bench.cpu</code> , <code>bench.mem</code> , <code>bench.disk</code> , <code>bench.net</code> , <code>bench.all</code> , <code>bench.contention</code>	benchmarks
<code>ioreg.*</code>	<code>ioreg.find</code> , <code>ioreg.node</code> , <code>ioreg.watch</code>	the I/O registry
<code>drives.*</code>	<code>drives.list</code> , <code>drives.set_target</code>	storage devices

zgo – the launcher

The launcher's surface is the thinnest of the census's named apps: system commands — sleep, lock, empty trash, and the rest — under `sys.*`, alongside the launcher's own workflow and snippet machinery. It is the app a script reaches for when it wants the *machine* to do something rather than an app: a run that ends by locking the screen, or a maintenance script that empties the trash, calls `zgo` for the one system verb it needs and no more.

The coreless four

`zphoto`, `zstation`, `ztranslator`, and `zcontainer` carry the bus but register no rich frontend catalog, so their enumerable surface is the shared `appshell.*` verbs plus their Tauri backend commands. A script drives them through the shared shell verbs above and whatever backend commands each exposes; the catalog reports exactly that surface for them and, honestly, no more. The reference names them here so the map is complete — every app that answers on a socket appears — while marking clearly that their scriptable depth is the shell's, not a domain catalog's, until they register one.

Using the reference

The reference is a map, not the territory. When a script needs the exact, current verb of an app, it does not consult this chapter or the catalog — it asks the running app with `verbs()`, and reads the answer the app will itself honor a moment later. This chapter teaches the shape so a reader knows which namespace to look in and roughly what to expect there; `verbs()` gives the precise, live, drift-free truth. That is the whole relationship the book has argued for from Chapter 3 onward: a surface that documents itself needs no second document to be correct, and this reference is offered in that spirit — a human's overview of a machine-readable truth, useful precisely because it never pretends to be the authority the live surface is.

Chapter 27 — State Queries in Depth

The third operation, reconsidered

Of the four operations, state reading is the one a new script author most often uses wrongly, not because it is hard but because its boundary with `call` is subtle. A verb *does* something and returns the result of doing it; a state query *reads* something that is already true and returns it without changing anything. The distinction is not enforced by the type system — both travel the same request/response substrate, both come back as a value — but it is real in the app's own contract, and honoring it is what keeps a script's behavior predictable. `library.search` is a verb: it runs a query and returns matches, and calling it twice runs the query twice. `selection` is a state query: it reports which items are selected right now, and reading it twice reads the same live fact twice with no side effect. When a script wants “what is true,” it reads state; when it wants “make this happen and tell me what happened,” it calls a verb. Getting this right is the difference between a script that observes an app and a script that keeps nudging it.

When to read state and when to call a verb

The decision rule is short and worth stating flatly. Read state when the answer is a property of the app's current condition that the app maintains anyway — the selection, the active collection, the connection status, the active environment. Call a verb when the answer requires the app to *compute* or *act* — to run a search, send a request, render a bibliography, open a document. The tell is idempotence: reading a state query twice in a row must return the same answer if nothing else changed, and if a script finds itself relying on a “read” that alters what the next read returns, it has mistaken a verb for a state query and the app's contract will eventually surprise it. State is the app's condition; verbs are the app's behavior; and a script that keeps the two straight reads cleanly and acts deliberately, never confusing the two.

The get path, mechanically

`get` runs the identical substrate as `call`, aimed at the state table instead of the verb table, which Chapter 4 established and this section makes concrete. A `get` frame — `{"t": "get", "id": 2, "state": "selection"}` — arrives, the handler resolves `selection` against the registered `state` entries rather than the verbs, runs the entry's `get` closure, and composes the same `{"t": "reply", "id": 2, "ok": true, "value": ...}` a verb would. There is no separate machinery: the state table is a second dictionary consulted by a second frame type, and everything downstream — correlation by `id`, projection of the result to a stryke value, the failure model — is shared with `call`. A state query that does not exist is rejected exactly as an unknown verb is, with `no such state: <name>`, because the allow-list discipline is the same. The economy is the point: an app that wired the one substrate got state reads for the cost of declaring a `state` array, and a script that learned `call` already knows `get` because they differ only in which table the name resolves against.

Reading state in a loop: the poll that knows

The tunnel recipe of Chapter 16 showed the archetypal use of state — a poll that reads a live condition until it reaches the desired value — and the pattern generalizes to every “wait until” a script needs. The shape is

always the same: read a state query, test it, and either proceed or sleep and read again, bounded by a try count so a condition that never arrives becomes a death rather than a spin.

```
# wait until an app's state reaches a condition, or die trying
fn wait_for($h, $state, $pred, $max) {
  val $tries = 0
  loop {
    if $pred($h->get($state)) { return true }
    if ($tries += 1) > $max { die "${ $state } never satisfied" }
    App::sleep(250)
  }
}

val $vpn = App::open("ztunnel")
$vpn->call("vpn.connect", %{ profile => "work" })
wait_for($vpn, "status", fn ($s) { $s->{state} == "connected" }, 40)
```

The helper is ordinary stryke — no app knows it exists — and it turns any state query into a synchronization primitive. This is the return path of the bus doing what the old palette could not: a fire-and-forget action followed by a blind sleep only hopes; a poll against a live state query knows, and the difference is load-bearing wherever a later step depends on an earlier one having taken effect.

State versus event: two answers to “did it change?”

State queries and events are the two ways a script learns that something changed, and they answer the question from opposite ends. A state query answers by *pull*: the script asks, at a moment of its choosing, what is true now, and gets the current answer. An event answers by *push*: the app tells the script the instant a thing happens, and the script receives it at the next pump. A poll built on a state query is simple and needs no subscription, but it only sees the condition at the moments it happens to read, and it costs a call per read. An event sees every occurrence and costs nothing between them, but it needs a subscription and a pump. The choice is the ordinary polling-versus-eventing tradeoff, and the bus offers both because both are right sometimes: poll a state query when the script controls the cadence and wants a specific condition, subscribe to an event when the script must not miss an occurrence and does not know when it will come. The tunnel poll wants the former; the download watcher of Chapter 16 wants the latter; and a script author who knows which question they are asking reaches for the right one.

Chapter 28 – The Frame Reference

The wire, exhaustively

Chapter 6 showed the wire by example; this chapter is the authoritative catalog of every frame, because a reader debugging a transcript or implementing a client needs the complete set in one place. Every frame is a single line of JSON, an object with a `t` field naming its kind, and there are exactly seven kinds — four a client sends and three a server sends. Nothing else ever appears on the wire.

Client-to-server frames

A client — the `App` module, in-process or over a socket — sends four frame kinds, and each carries an `id` the client mints to correlate the reply.

`call` invokes a verb.

```
{"t":"call","id":1,"verb":"library.search","args":{"q":"graphene"}}
```

`verb` is the verb id; `args` is an object of the typed arguments. The server validates `args` against the verb's declared `params` before running it and replies with one `reply` frame carrying the value or an error.

`get` reads a state query.

```
{"t":"get","id":2,"state":"selection"}
```

`state` is the state-query id; there are no arguments. The server runs the query's `get` and replies with one `reply` frame.

`verbs` requests the introspection manifest.

```
{"t":"verbs","id":3}
```

No further fields. The server replies with one `reply` frame whose `value` is the stripped manifest — the verbs, state, and events with their types but no function bodies.

`sub` opens a subscription.

```
{"t":"sub","id":4,"event":"itemAdded"}
```

`event` is the event id. The server replies with one `reply` frame confirming the registration, after which it pushes an `event` frame per occurrence, each tagged with this frame's `id` as its `sub`. A matching `unsub` — `{"t":"unsub","id":4}` — ends the subscription and stops the pushes.

Server-to-client frames

A server — an app's bridge — sends three frame kinds.

`reply` answers exactly one request, correlated by `id`.

```
{"t":"reply","id":1,"ok":true,"value":[{"id":42,"title":"..."}]}
{"t":"reply","id":9,"ok":false,"error":"no such verb: foo.bar"}
```

`ok` is the fork: `true` carries a `value`, `false` carries an `error` string. Every `call`, `get`, `verbs`, and `sub` produces exactly one `reply`, and the `id` is the request's `id`.

`event` delivers one occurrence of a subscribed event.

```
{"t":"event","sub":4,"event":"itemAdded","payload":{"id":43,"title":"..."}}
```

`sub` is the subscription's `id` — the `id` from the `sub` frame that opened it — and it is how the `App` module routes the payload to the right closure. `payload` is the value the app passed to `emit`. Event frames carry no `id` of their own, because they are not replies to a request; they are pushes correlated to a subscription.

There is no third distinct server frame in version one; the taxonomy is `reply` and `event`, and the two failures the wire cannot phrase — a broken pipe and a timeout — produce no frame at all, which is precisely why the `App` module manufactures a death for each rather than waiting for a frame that will never come.

A full transcript, annotated

Reading the frames in sequence is the fastest way to internalize the protocol. Here is a whole session — enumerate, read, call, subscribe, receive, fail — as it crosses one socket:

```
-> {"t":"verbs","id":1}
<- {"t":"reply","id":1,"ok":true,"value":{"verbs":[...],"state":[...],"events":[...]}}

-> {"t":"get","id":2,"state":"activeCollection"}
<- {"t":"reply","id":2,"ok":true,"value":"graphene-review"}

-> {"t":"call","id":3,"verb":"item.add","args":{"doi":"10.1/x"}}
<- {"t":"reply","id":3,"ok":true,"value":{"id":51,"title":"..."}}

-> {"t":"sub","id":4,"event":"itemAdded"}
<- {"t":"reply","id":4,"ok":true}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"id":52,"title":"..."}}
<- {"t":"event","sub":4,"event":"itemAdded","payload":{"id":53,"title":"..."}}

-> {"t":"call","id":5,"verb":"item.frobnicate","args":{}}
<- {"t":"reply","id":5,"ok":false,"error":"no such verb: item.frobnicate"}

-> {"t":"unsub","id":4}
```

Every reply's `id` matches its request's; every event's `sub` matches the subscription's `id`; the one failure is an `ok:false` naming the fault. The whole protocol is visible in those fourteen lines, and a client that handles exactly these frames handles the entire bus.

Why the frame set is closed

It is deliberate that there are seven frame kinds and not eight, and the discipline is the same one the versioning chapter named. A new *capability* — pipelining, batching, a source-side event filter — is added by putting a new *field* on an existing frame, which old readers ignore, not by inventing a new frame kind, which old readers would reject. So the frame set is closed by intent: the wire grows by widening frames, never by multiplying them, and a reader written against these seven kinds keeps working across every additive change the contract makes. The closed set is not a limitation; it is the guarantee that lets the protocol evolve without a flag day.

Chapter 29 – The Error Reference

Every way a call can fail, with its cause and its fix

The failure model of Chapter 14 gave the taxonomy; this chapter is the desk reference — every error a script can meet, what produced it, and what to do about it. The organizing fact is that all of them arrive as a death at the call site, so the message is the primary diagnostic, and knowing the message forms turns a stack trace into a diagnosis.

no such verb: <name>

Cause. A `call` named a verb in neither the engine's `COMMANDS` nor the frontend's registered set. Almost always a typo in the verb id, a verb that was renamed, or a verb the script assumed exists but the app never registered.

Fix. Read `verbs()` and check membership. The verb id you want is in the manifest or it is not; if it is not, either the name is wrong or the app does not expose it. A script that probes for optional verbs tests `verbs()` rather than calling and catching, because the manifest is the authority on what exists.

no such state: <name>

Cause. A `get` named a state query the app did not register. The state tables are smaller than the verb tables, so a mistaken state name is a common early error.

Fix. Read `verbs()->{state}` for the state queries the app actually exposes. If the value you want is not a state query, it may be the return of a verb instead — the selection is state, but the results of a search are a verb's return, and confusing the two produces this error.

missing required argument: <name> and the signature

Cause. A `call` omitted a parameter the verb declared `required`. The bridge rejects it before `run` is entered, and the error names the missing parameter and prints the verb's full signature.

Fix. Supply the argument. The signature in the message shows every parameter, its type, and which are required, so the fix is usually visible in the error itself. This is why the bridge validates against the manifest rather than letting the call reach `run` with a hole in it: the failure names exactly what is missing at the boundary, not three statements later inside the app.

<app> is not running (the undialed call)

Cause. `App::open("<app>")` failed at `connect` because no socket file for that app exists — the app is not up, or crashed without cleanup and has not relaunched. Per Chapter 10's second rule, the target must be running, and version one has no launch-on-demand.

Fix. Start the app, or gate the script on `App::list()` containing the name before opening it. A script that must tolerate an absent target checks the list first rather than opening and catching, because the list is the authority on what is dialable right now.

<app> vanished (the broken pipe)

Cause. An out-of-process target exited in the middle of a call; the pipe broke and no `reply` is possible. Per Chapter 14, the `App` module holds the connection and retries once on a broken pipe, so this death means the retry also failed — the target is genuinely down, not merely restarting.

Fix. This is usually a real condition to report, not a bug to work around: the app the script was driving died. A long-running script that expects targets to restart wraps the vulnerable calls in a `try` and re-opens the handle on failure, but a script that meets an unexpected `vanished` has learned something true about the world and should say so.

<verb> timed out

Cause. The target accepted the call and never replied under the deadline — a `run` body awaiting a promise that never settles, an engine call that wedged. Distinguished from a broken pipe by the connection staying open, per Chapter 23's fingerprints.

Fix. This points at the verb's implementation, not the script: a verb that reliably times out is a verb with a bug in its `run`. The script's only recourse is to catch the timeout and treat the verb as unavailable; the real fix is in the app, and the timeout death is the bus refusing to let the app's hang become the script's hang.

Sending a non-serializable value

Cause. A `call` tried to pass a stryke value with no JSON image — a closure, a handle, or a container holding one — as an argument. Per Chapter 13, these are refused at the boundary rather than serialized into something broken.

Fix. Do not pass behavior or connections as data. A callback belongs in `on`, which never serializes it; a handle stays VM-local and the script reads through it rather than handing it across. If a map carries a closure by accident, the fix is to send only its data fields.

The one error a script should never see

There is a failure the design works to make impossible rather than to report: a silent wrong answer. Every error above is loud — a death at the call site, with a message naming the fault — precisely so that the failure mode a script author most fears, a call that appears to succeed but returns the wrong thing, does not arise from the bus itself. The bus can deliver an app's own bug faithfully — a verb whose `run` returns the wrong value is the app's fault, surfaced when a script reads a field that is not there — but the bus's own layers do not silently corrupt: a call either returns the value the app produced, or it dies with a named cause. That guarantee is the sum of the whole failure model, and it is why the error reference is a list of loud, diagnosable deaths and not a warning about quiet wrong answers.

Chapter 30 – Cookbook II: Reporting, Maintenance, Migration

The scripts that run on a schedule

The first cookbook showed the bus moving values between apps in a single interactive run; this one collects the scripts that earn their place by running unattended — a report generated nightly, a maintenance pass run weekly, a migration run once. They lean harder on the failure model, because a script no one is watching must fail loudly and finish cleanly, and they lean on state queries and events to know when their work is done.

A cross-app status report

The most useful unattended script is often the plainest: gather one fact from each running app and print a report. It uses only `App::list` and `verbs/get`, touches every app generically, and never assumes a verb an app might not have:

```
#!/usr/bin/env stryke
use App

p "=== bus status ${\ now() } ==="
for val $name (@{ App::list() }) {
  val $h = App::open($name)
  val $surface = $h->verbs()
  val $nverbs = scalar @{ $surface->{verbs} }
  val $nstate = scalar @{ $surface->{state} }
  val $nevents = scalar @{ $surface->{events} }
  p "${ $name }: ${ $nverbs } verbs, ${ $nstate } state, ${ $nevents } events"
}
```

The report is generic by construction: it drives every app through the shared seven calls and asks each only for its own manifest, so it works against an app it has never seen and grows automatically as apps join the bus. This is the census of Chapter 8 rendered live — not read from a generated file but asked of the running apps, one `verbs()` at a time.

A maintenance pass with per-app tolerance

A weekly maintenance run touches several apps and must not let one app's failure abort the rest. The pattern is a loop over the apps, each unit of work wrapped so a failure is recorded and the pass continues:

```
#!/usr/bin/env stryke
use App

val @report = @{}

fn step($label, $fn) {
  val $r = try { $fn(); "ok" } catch ($e) { "FAIL: ${ $e }" }
  push @report, "${ $label }: ${ $r }"
}

# each app's maintenance, isolated from the others
if grep(App::list(), "zemail") {
  val $mail = App::open("zemail")
  step("mail: expunge trash", fn () { $mail->call("folder.digest", %{ folder => "Trash"
  }) })
}
if grep(App::list(), "zcite") {
  val $cite = App::open("zcite")
  step("cite: lint collections", fn () { $cite->call("collection.lint", %{ }) })
}
if grep(App::list(), "zthrottle") {
  val $thr = App::open("zthrottle")
  step("throttle: snapshot procs", fn () { $thr->call("proc.snapshot", %{ }) })
}

p "=== maintenance report ==="
for val $line (@{ @report }) { p $line }
if grep(@report, qr/FAIL/) { die "maintenance had failures" }
```

Three properties make this fit for unattended use. It checks `App::list()` before opening each app, so a target that happens to be down is skipped rather than aborting the run at `connect`. It isolates each unit of work in `step`, so one app's failure is a line in the report, not the end of the pass. And it ends in a non-zero exit when any step failed, so a scheduler that runs it sees the failure without a human reading the log. The bus's failure model — a death by default, a `try` where the author means to continue — is exactly the substrate this pattern needs.

A one-time migration between two apps

A migration run moves data from one app's world into another's, once, and it wants a dry run before it commits. Here reading-list entries from the browser become library items in the reference manager, with a preview pass that changes nothing:


```
#!/usr/bin/env stryke
use App

val $dry = App::env("DRY_RUN") == "1"
val $wire = App::open("zwire")
val $cite = App::open("zcite")

val @list = @{ $wire->get("readingList") }
p "reading list holds ${\ scalar @list} entries (dry_run=${ $dry })"

val $added = 0
for val $entry (@{ @list }) {
  val $doi = doi_from_url($entry->{url})      # ordinary stryke helper
  if !$doi { next }
  if $dry {
    p "would add: ${ $doi }"
  } else {
    val $item = $cite->call("item.add", %{ doi => $doi })
    $cite->call("collection.add_item",
      %{ collection => "migrated", item => $item->{id} })
    $added += 1
    p "added: ${ $item->{title} }"
  }
}
p "${ $dry ? "would add" : "added" } ${ $dry ? scalar @list : $added } items"
```

The dry-run switch is the migration's safety: run it once with `DRY_RUN=1` to see exactly what it would do, read the preview, then run it for real. The bus makes the preview cheap because reading is free of effect — `get("readingList")` observes without changing, and `doi_from_url` is pure — so the dry run is the real run with its two mutating calls skipped. This is the plan-then-act shape of the transfer recipe generalized: a migration that can be inspected before it commits is a migration a person can trust to run once and be done.

A watcher that turns events into a log file

The last recipe is a long-running watcher that subscribes to several apps at once and writes every event to a single log — the bus as an observability tap across the suite:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $wire = App::open("zwire")

$cite->on("itemAdded", fn ($it) { logline("zcite", "itemAdded", $it->{title}) })
$wire->on("downloadComplete", fn ($dl) { logline("zwire", "downloadComplete", $dl->{path})
  ~ })

fn logline($app, $event, $detail) {
  append_file("~/bus-events.log", "${\ now() }\t${ $app }\t${ $event }\t${ $detail }\n")
}

# drain both streams forever, one pump serving every subscription
loop { App::pump(); App::sleep(500) }
```

One script, two apps, two event streams, one log. `App::pump()` drains every subscription the script holds, regardless of which app or which handle it belongs to, so a single pump loop serves the whole tap. The watcher never polls an app and never asks a verb; it sits on the events the apps already emit and folds them into a file, and it is the plainest demonstration that the bus's event half is not an afterthought but a peer of its call half — a

script can be built entirely out of watching, with not a single `call` in it, and still do real and useful work.

The through-line of both cookbooks

Across every recipe in both cookbooks, one property recurs and is worth naming as the book closes its practical half: the script is always the only place two apps' values meet. The bus never lets one app hold a connection to another; it makes each app a server to the user's scripts and nothing more, so a value that leaves `zcite` and enters `zreq` passes through a stryke variable in between, in a program the user wrote and can read top to bottom. That is the shape the opening sentence promised and the whole book has been building toward — one language driving every app by name, each app's answer landing in a variable the next call can use — and the cookbooks are simply that sentence, spent as many ways as a working day has problems.

Chapter 31 – Implementing the Handler

The shared crate, from the app's side

The tutorial of Chapter 19 gave the app author two lines and moved on; this chapter is for the reader who wants to know what those two lines rest on — the `zgui-bridge` crate, seen from inside. The crate exists so that no app writes socket code, frame code, or correlation code, and understanding its three pieces — `serve`, the `Bridge`, and the `Handler` — is understanding everything an app author does not have to build. The design goal is that the crate owns everything generic and the app owns only what is specific to it, which turns out to be a single decision per call: does this verb belong to the engine or to the frontend?

`serve`: the front door

`serve(app_name, handler)` is the whole of the socket lifecycle, and it does in order the boring, correct things Chapter 6 named. It computes the socket path from `app_name` and the runtime directory; it creates the `zgui/` parent if missing; it unlinks any stale socket file so a crashed predecessor's corpse does not block the bind; it binds a fresh `UnixListener` at mode `0600`; and it enters an accept loop, spawning a task per accepted connection. Each task owns one connection's state — its correlation table, its subscriptions — and reads frames in a loop, dispatching each through the `Bridge` to the app's `Handler` and writing back the reply. When the connection drops, the task tears down that connection's state and exits, which is the clean-disconnect teardown Chapter 20 leans on. The app author writes none of this; they pass a name and a handler and get a served, mode-restricted, crash-recoverable socket with per-connection isolation.

The `Handler`: one decision, two arms

The `Handler` is the one interface the app implements, and its core is the routing decision of Chapter 7 expressed as code the author fills in. Given a verb name and arguments, the handler answers whether the engine owns the verb and, if so, calls the engine directly; otherwise it forwards to the frontend. In shape:

```
impl Handler for MyApp {
    fn is_engine_cmd(&self, cmd: &str) -> bool {
        core::commands::COMMANDS.contains(&cmd)
    }

    async fn call_engine(&self, cmd: &str, args: Json) -> Reply {
        // route straight to the -core engine, in Rust, no webview
        core::dispatch(cmd, args).into()
    }

    async fn call_frontend(&self, cmd: &str, args: Json) -> Reply {
        // evaluate the dispatch shim in the webview and await the reply channel
        self.webview_dispatch(cmd, args).await
    }
}
```

The `Bridge` calls `is_engine_cmd` once per incoming `call` and dispatches to whichever arm it selects, composing the `reply` frame from the returned `Reply` and correlating it by the request's `id`. The author's

whole responsibility is those three methods, and two of them are nearly mechanical: `is_engine_cmd` is a set membership test against a list the core already exports, and `call_engine` is a call into the engine the UI already uses. Only `call_frontend` carries real machinery, and even that is the reply-channel pattern the crate provides a helper for.

The reply channel, from the backend

`call_frontend` is where the correlation of Chapter 4 lives in the app's own code, and it is worth seeing whole because it is the one genuinely asynchronous seam an app author touches. The backend cannot call into the webview and get a return value synchronously — the webview runs its verb asynchronously and reports back through a separate Tauri command — so the backend registers a one-shot channel keyed by the request `id`, evaluates the dispatch shim passing that `id`, and awaits the channel:

```
async fn webview_dispatch(&self, cmd: &str, args: Json) -> Reply {
    let id = self.next_id();
    let (tx, rx) = oneshot::channel();
    self.pending.lock().insert(id, tx); // stash the reply slot
    self.window.eval(&format!(
        "window.__zguiBridgeDispatch({id}, 'verb', {cmd:?}, {args:?})");
    rx.await.unwrap_or(Reply::err("frontend dispatch dropped"))
}

// the Tauri command the shim calls back into, fulfilling the slot
#[tauri::command]
fn zgui_bridge_reply(state: State<MyApp>, id: u64, ok: bool, value: Json) {
    if let Some(tx) = state.pending.lock().remove(&id) {
        let _ = tx.send(if ok { Reply::ok(value) } else { Reply::err_json(value) });
    }
}
```

The `id` is the thread through the whole loop, exactly as Chapter 4 described from the outside: minted at the call, carried into the webview by the shim, carried back by `zgui_bridge_reply`, and used to find the one waiting slot. The `oneshot` channel is the concrete form of “a per-request channel that fulfills exactly once” — send once, receive once, and a dropped sender (the shim failed or the webview reloaded) surfaces as a clean error rather than a hang, because `rx.await` on a dropped `tx` resolves to the fallback, not to nothing.

What the crate refuses to own

A shared crate earns trust by the boundaries it declines to cross, and `zgui-bridge` draws one deliberately: it owns the transport and the correlation, and it owns nothing about what a verb *means*. It does not know that `library.search` searches a library or that `vpn.connect` raises a tunnel; it moves the frame to the handler and the reply back, and the semantics live entirely in the app. This is why one crate serves eleven apps of wildly different domains without a line of domain code in it: the crate is a pipe with a correlation table, and the meaning is always the app's. The same discipline that made the `App` module seven calls with the app's verbs on top makes the host one crate with the app's handler on top — a thin, generic substrate under a domain surface, top to bottom, which is the whole architecture in one sentence.

Chapter 32 – Cookbook III: Interactive and Composed Sessions

The bus at the prompt

The first two cookbooks wrote whole files; this one works at the prompt, because a large part of the bus's daily value is ad hoc — a question asked of a running app in the middle of doing something else. The REPL of Chapter 17 is the vehicle, and its idioms are worth collecting, because an interactive session composes the same seven calls into something closer to a conversation than a program.

Discovering an app you have never scripted

The first thing to do with an unfamiliar app is ask it what it can do, and the REPL turns that into a two-line habit. Open the app, print its verbs filtered to the namespace you suspect, and read the signatures:

```
val $h = App::open("zftp")
for val $v (@{ $h->verbs()->{verbs} }) {
  if $v->{id} =~ /^sync\.\/ {
    p "${ $v->{id} }({{ join(", ", @{ $v->{params} } |map| "${ .{name} }": ${ .{type} }" ) }}) -> ${ $v->{returns} }"
  }
}
```

The output is the exact `sync.*` surface the app will answer to, with types, generated from the live manifest. A script author who does this before writing a line of automation starts from what the app actually exposes rather than from a guess, and the habit is cheap enough — two lines at a REPL — to make it the default first move against any app.

Building a call incrementally

An interactive session lets a call be built up rather than written whole, testing each piece against the real app. Read the state first, inspect it, then use it:

```
val $req = App::open("zreq")
val $env = $req->get("activeEnvironment")          # what is active?
p $env                                             # inspect it
val @hist = @{ $req->call("history.list", %{ limit => 5 }) }
for val $h (@{ @hist }) { p "${ $h->{method} } ${ $h->{url} } -> ${ $h->{status} }" }
val $again = $req->call("history.replay", %{ id => @hist->[0]->{id} }) # re-run the last
p "replayed: ${ $again->{status} }"
```

Each line is checked against the live app before the next is written, so a session grows into a script by accretion — the parts that worked, kept; the parts that did not, discarded — and what remains is a correct automation assembled from verified pieces rather than a hopeful one written all at once and debugged after.

Composing two apps at the prompt

Even cross-app composition works interactively, because a handle held at the REPL is a live connection for as long as the session lasts. Open two apps, read from one, and feed the other, watching each result:

```
val $cite = App::open("zcite")
val $req  = App::open("zreq")
val @sel  = @({ $cite->get("selection") }) # what is selected in zcite?
p "selected ${\ scalar @sel} items"
val $it   = @sel->[0]
val $res  = $req->call("request.send",
  %{ url => "https://api.crossref.org/works/${ $it->{doi} }" })
p "crossref says: ${ $res->{status} }"
```

This is the invoice pipeline of Chapter 16 discovered rather than written — each seam tried by hand, each value inspected before it is passed on — and it is the way most real orchestrations are found: interactively, one verified handoff at a time, then saved to a `.stk` file once the whole chain works. The REPL is not a lesser mode of the bus; it is where scripts are born.

A composable helper library

Interactive discovery eventually crystallizes into helpers a script author reuses, and the bus's uniformity makes them app-agnostic. A handful of small functions, written once against the seven calls, work against every app:

```
# does an app expose this verb?
fn has_verb($h, $id) {
  for val $v (@{ $h->verbs()->{verbs} }) { if $v->{id} == $id { return true } }
  false
}

# call a verb only if it exists, else nil – tolerate surface differences across builds
fn call_if($h, $id, %args) {
  if has_verb($h, $id) { $h->call($id, %args) } else { nil }
}

# every running app that exposes a given verb
fn apps_with($id) {
  grep(map(App::list(), fn ($n) { %{ name => $n, has => has_verb(App::open($n), $id) }
    ~  })),
    fn ($x) { $x->{has} })
}
```

None of these knows a single app-specific fact; they are built from `verbs`, `call`, and `list`, and they work against `zcite`, `zreq`, or an app written next year, because the uniformity of the surface is what lets a helper be generic. This is the platform property of Chapter 18 spent at the script level: because every app answers the same seven calls, a utility written against those calls is a utility for the whole suite, and a script author's personal library of helpers grows to cover apps it was written before they existed.

Chapter 33 – The Bus and the Connector Family

App among its siblings

The `App` module did not arrive alone; it joined a family, and seeing it beside its siblings is the last thing needed to place it. `stryke`'s connector packages — `stryke-gui`, `stryke-aws`, and now `stryke-app` — share one shape: a `cdylib` with C-ABI exports and a `stryke`-facing module on top, a foreign capability made native to the language. The family resemblance is not incidental. It is the discipline that let `stryke-app` be built quickly and be trusted immediately, because it is the third instance of a pattern already proven twice, and a reader who knows one connector knows how to read the others.

`stryke-gui`: the physical sibling

`stryke-gui` is the connector the `App` module is most usefully contrasted with, because the two are complementary rather than competing, and the contrast sharpens what the bus is. `stryke-gui` is *physical*: it moves the mouse, presses keys, drives the screen — it automates a GUI the way a human's hands would, from the outside, against pixels and input events. `App` is *semantic*: it calls named verbs and reads typed state — it automates an app the way the app's own code would, from the inside, against functions and values. Where `stryke-gui` clicks the button labeled "Search," `App` calls `library.search` and gets the rows. The two share the same connector shape and the same performance discipline — `stryke-gui` holds one persistent input handle rather than spawning a tool per keystroke, exactly as the bus holds one socket rather than forking per call — but they reach an app at two different layers. `stryke-gui` works against any app, including ones that never joined the bus, at the cost of driving a surface meant for human eyes; `App` works only against apps on the bus, but reaches their actual functions and returns. A script that needs both — drive an un-bussed app's UI with `stryke-gui`, then hand its result to a bussed app with `App` — uses them together, two connectors of one family solving two halves of one automation.

`stryke-aws`: the remote sibling

`stryke-aws` is the sibling that shows the family is not about locality. It reaches a remote cloud API — authenticated, over the network, against someone else's service — and it wears the same connector shape as the two local siblings. Placing it beside `App` draws the boundary the bus draws for itself in Chapter 18: `stryke-aws` is a remote connector by design, with the network's threat model — credentials, transport security, capability scoping — baked into what it is, while `App` is a local connector by design, refusing the network precisely because it has no threat model for it and does not pretend to. The family holds both a remote member and a local member without contradiction, because each was built for its own boundary, and the bus's refusal to grow a network bind is not a limitation of the connector pattern but a choice about which member of the family `App` is. Cross-machine automation, if it ever ships, would be a different connector with `stryke-aws`'s posture, not `App` with a wider bind.

The precedent from `zterminal`

One more relative sits outside the connector family but inside the same discipline: `zterminal`, which the transport chapter named as the socket precedent. `zterminal` speaks a raw wire protocol — the tmux `imsg` protocol — straight to a socket, with no subprocess between, and it did so before the bus existed. Its relevance is that it proved the exact discipline the bus's out-of-process transport needed: that a MenkeTechnologies app can hold a persistent socket and speak a framed protocol over it natively, without shelling out to a tool per operation. The bus did not invent that capability; it pointed a second protocol — newline JSON instead of `imsg` frames — down a pipe of the same kind `zterminal` had already dug. Naming the precedent is the book's consistent method: nothing here is invented where something could be reused, and the socket transport is reused discipline, not new discipline.

Where the family points

Seen whole, the connector family sketches the shape of stryke's reach: physical automation of any GUI (`stryke-gui`), semantic automation of the bussed suite (`stryke-app`), remote automation of cloud services (`stryke-aws`), each a `cdylib-plus-module` built to one shape, each native to the language, none a subprocess spawned per call. The bus is the family member that turns the author's own suite into a programmable surface, and it is the one that could only exist because the suite was built the way it was — thin shells over shared cores, each already exposing a command surface to its own frontend. The other connectors reach outward, to foreign GUIs and foreign clouds; `App` reaches inward, to the functions the author already wrote, and hands them a caller. That inward reach is the whole book: not a new capability bolted onto the apps, but the recognition that a suite of shared-core apps is already a single programmable surface, wanting only the small set of pieces — a typed manifest, a correlated bridge, a stryke module, a newline of JSON — that let one language call every function in it, by name, and get the answer back.

Chapter 34 — Patterns and Anti-Patterns

The idioms worth naming

A capability accretes idioms — recurring shapes that experienced users reach for without thinking — and a book that has shown the bus in use can now name them, because naming an idiom is what lets a reader recognize it in their own code and reach for it deliberately. The patterns below are all present in the recipes of the earlier chapters; this chapter lifts them out and states each as a shape, with the anti-pattern it replaces, so that a script author meets each once as a named thing rather than rediscovering it under deadline.

The star topology

The foundational pattern is the one Chapter 13 proved falls out of the handle rule: the script is always the hub, and every app is a spoke. A script that drives four apps holds four handles and passes every cross-app value through its own variables; no app ever holds a connection to another. The shape is a star with the script at the center, and it is worth adopting deliberately rather than merely inheriting, because it is what makes an orchestration readable — every place two apps' data meets is a line in the script, visible, inspectable, and yours. The anti-pattern it replaces is the one the handle rule makes impossible: handing app A a handle to app B and stepping back, hoping the two apps talk among themselves. They cannot, and the reason is a feature — a script you can read top to bottom is worth more than a mesh you cannot.

Poll-until for state, subscribe-for-events

The second pattern is the poll-until of Chapter 27: read a state query in a bounded loop until it reaches the desired value, then proceed. Its partner is the subscribe-and-pump of Chapter 15: register a callback and drain it at a controlled cadence. The pattern is choosing correctly between them — poll a state query when the script controls the cadence and waits for one condition; subscribe to an event when the script must not miss an occurrence. The anti-pattern both replace is the *blind sleep*: firing an action and then `App::sleep`-ing a guessed interval in the hope that the action finished. A blind sleep is wrong in both directions at once — too short and the next step runs before the action took effect, too long and the script wastes the difference on every run — and the return path the bus adds exists precisely to replace hoping with knowing. Any `App::sleep` not inside a poll loop bounded by a state read is a smell worth a second look.

Try-per-item for batches

The third pattern is the try-per-item of Chapters 14 and 16: in a loop over a batch, wrap each unit of work so a failure becomes a recorded outcome rather than the end of the run. The shape is a `try/catch` inside the loop body, pushing successes to one list and failures to another, and finishing with a report. The anti-pattern it replaces is the bare loop that dies on the first bad item, losing every item after it and telling the user nothing about which one failed or why. A batch script that must process every item it can and report the rest is the common case for unattended work, and the try-per-item is its shape. The discipline is to make the default — a death that stops the script — opt out of, per item, only where the author genuinely means to continue.

Discover-before-call

The fourth pattern is the discover-before-call of Chapters 17 and 32: ask `verbs()` what an app exposes before writing calls against it, and, in code that must tolerate surface differences across builds, test membership before calling. The `call_if` helper of Chapter 32 is this pattern crystallized — call a verb only if it exists, else nil. The anti-pattern it replaces is *fishing by calling*: probing for a verb's existence by calling it and catching the `no such verb` death. Fishing works but it is backwards — it uses the failure path as a control-flow branch, which is exactly the confusion Chapter 14 warned against — and `verbs()` gives the answer directly, cheaply, and without a death to catch. Probe the manifest; do not fish the dispatcher.

The pure-helper seam

The fifth pattern is subtle and Chapter 16 named it in passing: ordinary stryke computation sits in the same straight-line program as app calls, and keeping the two visibly separate is what makes a script readable. `doi_from_url` is a pure helper; `item.add` is an app call; and a script reads cleanly when the reader can see at a glance which lines touch an app and which are local computation. The anti-pattern is pushing pure computation into app verbs that do not need to be app verbs — asking an app to parse a string a stryke function could parse — which spends a socket round trip on work the VM could do for free and blurs the line between what the script computes and what the apps provide. The rule is: compute in stryke what stryke can compute, and call an app only for what only the app knows.

The single-pump loop

The sixth pattern is the single-pump loop of Chapter 30's event tap: one `App::pump()` in one loop drains every subscription the script holds, across every app, because pump is global to the VM's subscriptions, not per-handle. A watcher on three apps' events needs one pump loop, not three. The anti-pattern is a per-handle pump — imagining that each app's events need their own drain — which does not match the model and, worse, tempts a script into nested loops that starve one stream while draining another. One pump, one loop, all subscriptions: the model is simpler than the mistake.

The idempotent read

The last pattern is the discipline of Chapter 27: treat state reads as idempotent observations and verbs as effects, and never rely on a read to change what the next read returns. The anti-pattern is mistaking a verb for a state query — calling a search verb where a `get` of live state was meant, and then being surprised that “reading” it twice ran the query twice. The tell is the same one Chapter 27 gave: if reading a thing twice in a row does not return the same answer with nothing else changed, it is a verb, not a state query, and the script's model of the app is off by exactly that confusion.

Chapter 35 — Cookbook IV: Domain Recipes

Recipes that live inside one app

The earlier cookbooks favored cross-app orchestrations, because those are the recipes only the bus makes possible; this last cookbook returns to single-app automations, because much daily work is one app driven hard, and the bus makes that expressible too. Each recipe here spends one app's surface deeply rather than crossing between apps, and each is grounded in the namespaces the verb reference names.

Bulk-tagging a search result in the reference manager

A librarian's common chore is to run a search and apply a treatment to every hit — here, adding each result to a collection and re-exporting the whole collection in a citation style, all against `zcite` alone:

```
#!/usr/bin/env stryke
use App

val $cite = App::open("zcite")
val $query = "machine learning AND materials"

val @hits = @{ $cite->call("library.search", %{ q => $query }) }
p "search matched ${\ scalar @hits} items"

$cite->call("collection.add", %{ name => "ml-materials" })
for val $it (@{ @hits }) {
  $cite->call("collection.add_item",
    %{ collection => "ml-materials", item => $it->{id} })
}

# render the whole collection as a bibliography and write it to disk
val $bib = $cite->call("bib.bibliography",
  %{ collection => "ml-materials", style => "apa" })
write_file("~/ml-materials.txt", $bib->{text}) # ordinary stryke I/O
p "wrote ${\ scalar @hits} references"
```

The recipe touches three namespaces and never leaves the app, but it is still automation the old palette could not express, because each step consumes the previous step's return: the hits feed the collection, and the collection feeds the bibliography.

Regression-testing an API collection with assertions

A developer's chore is to run a saved request collection and assert on the responses — turning `zreq` into a test runner driven from a script rather than clicked through by hand:

```
#!/usr/bin/env stryke
use App

val $req = App::open("zreq")
$req->call("environment.activate", %{ name => "ci" })

val @cases = @{
  %{ id => "health", want => 200 },
  %{ id => "auth", want => 401 },
  %{ id => "listing", want => 200 },
}
val $fail = 0
for val $c (@{ @cases }) {
  val $res = try { $req->call("request.send", %{ id => $c->{id} }) }
    catch ($e) { $fail += 1; p "ERR ${ $c->{id} }: ${ $e }"; nil }
  if $res && $res->{status} != $c->{want} {
    $fail += 1
    p "FAIL ${ $c->{id} }: want ${ $c->{want} }, got ${ $res->{status} }"
  } elsif $res {
    p "ok ${ $c->{id} }: ${ $res->{status} }"
  }
}
if $fail > 0 { die "${ $fail } assertion(s) failed" }
p "all ${\ scalar @cases} cases passed"
```

The saved requests, the environment, and the send are all **zreq**'s own features; the assertions and the exit status are the script's. The bus is what lets the two meet — the response landing in **\$res** where the script can assert on its status — and the result is a test suite whose cases live in the API client and whose logic lives in a file a CI job runs.

Auditing a remote store against a policy

An operator's chore is to walk a remote tree and flag what violates a policy — here, files that are world-readable or larger than a bound, using **zftp**'s filesystem namespace:

```
#!/usr/bin/env stryke
use App

val $ftp = App::open("zftp")
val @flagged = @{}

fn walk($path) {
  for val $e (@{ $ftp->call("fs.list", %{ path => $path }) }) {
    if $e->{type} == "dir" {
      walk("${ $path }/${ $e->{name} }")
    } else {
      if $e->{size} > 100_000_000 {
        push @flagged, "${ $path }/${ $e->{name} }: large (${ $e->{size} })"
      }
      if $e->{mode} & 0o004 {
        push @flagged, "${ $path }/${ $e->{name} }: world-readable"
      }
    }
  }
}

walk("/srv/data")
p "flagged ${\ scalar @flagged} files"
for val $f (@{ @flagged }) { p "  ${ $f }" }
if scalar @flagged > 0 { die "policy violations found" }
```

`fs.list` returns each entry as data with a name, type, size, and mode, and the recursion, the policy, and the report are ordinary stryke. The app knows how to list a remote directory; the script knows the policy; and the audit is the two composed. Nothing here crossed apps, yet nothing here was possible before the bus, because before it there was no way to read a remote listing into a program and reason over it.

Driving the office engine as a document factory

The last recipe treats `zoffice` as a batch document factory: for each row of a data range, fill a template and render it, producing a stack of finished documents from one sheet and one template:

```
#!/usr/bin/env stryke
use App

val $office = App::open("zoffice")

# the data rows that drive the batch
val @rows = @{ $office->call("calc.cells",
  %{ sheet => "Certificates", range => "A2:C100" }) }

for val $row (@{ @rows }) {
  val $name = $row->{a}
  if !$name { next }
  # open the template, substitute, render to PDF
  $office->call("writer.open", %{ path => "~/templates/cert.odt" })
  $office->call("writer.replace", %{ find => "{{name}}", with => $name })
  $office->call("writer.replace", %{ find => "{{course}}", with => $row->{b} })
  $office->call("writer.replace", %{ find => "{{date}}", with => $row->{c} })
  val $out = $office->call("writer.render",
    %{ format => "pdf", path => "~/out/cert-${ $name }.pdf" })
  p "rendered ${ $out->{path} }"
}
p "produced ${\ scalar @rows} certificates"
```

`calc.cells` supplies the data, `writer.open/writer.replace/writer.render` drive the template, and the loop is the script's. The office engine has every one of these features for a human clicking through them one document at a time; the bus is what lets a script drive them a hundred documents at a time, each value from the sheet flowing into a substitution and out as a rendered file. It is the plainest statement of what the whole book has argued: the apps were always sets of functions waiting to be called, and the bus is the caller — for one app driven a hundred times, exactly as much as for a dozen apps driven once.

Chapter 36 — A Design FAQ

The questions the chapters raise but do not stop for

A book that argues a design linearly leaves questions in its wake — the “but what about” that a reader holds while the argument moves on. This chapter is for those, gathered and answered directly. Each answer is grounded in a decision an earlier chapter made; the FAQ form is a second index into the design, organized by the question a skeptical reader asks rather than by the layer the answer lives in.

Why not HTTP, or gRPC, or D-Bus?

Because the bus is a single-user, single-machine local facility, and each of those brings machinery it does not need and a threat surface it refuses. HTTP brings a network stack, request framing, and status semantics for a conversation that never leaves the machine; gRPC brings a schema compiler and a code-generation step for a wire that is already a newline of JSON a human can read; D-Bus brings a system bus, a broker process, and an introspection language for what a directory of socket files provides directly. The bus chose a Unix socket and newline JSON because they are the smallest thing that carries a correlated request/response with an attached stream, and because they keep the security model to one sentence — a file at mode `0600`, no network code, ever. Every heavier transport would add capability the bus does not want and a bind the bus refuses.

Why synchronous calls instead of async everywhere?

Because an automation script is almost always a sequence of steps where each depends on the last, and for that shape a synchronous call is both the simplest to write and the correct default for backpressure. `App::call` blocks until its reply, so the script cannot outrun the app, and a straight-line orchestration reads top to bottom as it executes. The asynchronous machine is still there underneath — a timer against a correlation id, per Chapter 14 — but it surfaces as a function that returns a value or dies, which is the API an automation author wants. A future pipelined mode can let a script fire a batch and gather the answers, and the correlation `id` on every frame was put there from version one precisely so that mode is an addition rather than a redesign; but the default is synchronous because the default use is sequential.

Why does a failed call die instead of returning nil?

Because a mistyped verb or a dead target is a bug in the script, not a branch the script meant to take, and returning nil would let the mistake propagate silently until some later line dereferenced the nil and blamed the wrong statement. Dying at the call site, with the fault named, puts the error where the fault is. A script that genuinely wants to survive a failure opts in with a `try`, per Chapter 14 — but the default is a loud death, so the fast path stays free of defensive ceremony and the slow path fails honestly. The single worst outcome for an automation script is a hang, which produces no signal at all; the second worst is a silent wrong answer; and the failure model spends its complexity to make both impossible, leaving only the loud, diagnosable death.

Why can a handle not be passed to another app?

Because a handle is a live connection, not a value, and there is nothing on the far side to receive a connection. Serializing a handle would mean shipping a socket, which means nothing to the app that received it. The rule that a handle is VM-local is what produces the star topology of Chapter 13 — the script stays the hub through which every value passes, and no app ever holds a connection to another — and that topology is also exactly what the security model wants, since an app is never a client of another app, only ever a server to the user's scripts. One decision, two chapters' worth of consequence, no coordination needed between them.

Why is there no “run arbitrary code” verb?

Because such a verb would be one entry in the allow-list that made every other entry meaningless. A caller with a generic eval verb could do anything the app's process could, and the careful enumeration of typed verbs would be decoration over a hole. The apps that legitimately run arbitrary code do it only through the palette's `js` and `bash` steps, which are not bus verbs — they are steps in a command the user typed into their own local palette, never reachable through `call`. The distinction is exact and load-bearing: an escape hatch the user reaches for deliberately in a local editor is a different thing from a capability a remote caller can invoke, and the security model protects precisely that line.

What happens if two copies of an app launch at once?

The second to bind wins the socket name, and the first is left serving a socket no dialer will find. Per Chapter 6, this is treated as correct rather than papered over, because the suite's apps are single-instance by intent — one running `zcite`, one command palette, one front door — so “the last to bind owns the name” is the single-instance rule expressing itself at the socket layer. A future launch-on-demand is where this would be tightened; version one lets the last writer win, and the unlink-before-bind step is what makes a crashed app's stale socket recoverable on the next launch.

Does carrying the bus slow an app down when nothing is driving it?

No, to a first approximation. At rest the bus is a bound `UnixListener` with no connections, a registered surface sitting as an in-memory data structure, and the `emit` calls the app makes at its own event points — each a membership check against an empty subscription list. There is no polling loop and no background thread; the listener blocks in the kernel waiting for a `connect` that may never come, which costs a file descriptor and nothing else. An app on the bus that no one is automating is, per Chapter 24, an app not on the bus at all, which is the property that let the fan-out reach every app without weighing the runtime cost of each.

Can a script drive an app it has never seen?

Yes, and the generic status report of Chapter 30 is the proof. A script can `App::list()` the running apps, `App::open` each, and `verbs()` its surface, learning what the app exposes at run time rather than from a hardcoded list. Helpers like `has_verb` and `call_if` from Chapter 32 are built entirely from the seven calls and work against any app, including one written after the helper. This is the platform property of Chapter 18 spent at the script level: because every app answers the same seven calls, a utility written against those calls is a utility for the whole suite.

Why version the wire, the surface, and the module separately?

Because they age at three different rates, per Chapter 22. The surface grows every time an app adds a feature; the module moves rarely because a hundred scripts import it; the wire moves slowest of all because everyone speaks it. A policy that treated them as one thing would either freeze the fast-moving surface to protect the slow-moving wire or destabilize the wire every time a verb was added. Versioning them separately lets the surface grow additively — new verbs, new optional parameters, all non-breaking — while the wire changes only by adding fields old readers ignore, which is why the frame set is closed and grows by widening frames rather than multiplying them.

When will cross-machine automation ship, and how?

If it ships, it ships as its own thing with its own threat model — authentication, transport security, capability scoping — not as this socket with a wider bind. Chapter 18 is firm on this: the bus's whole security posture rests on the socket being a single-user, single-machine artifact at mode `0600`, and the moment a frame can arrive from another host every verb becomes a remote procedure call the allow-list was never designed to distrust. Cross-machine is a real and different problem, and folding it into the local bus by loosening one bind would dissolve a clean model. The connector family of Chapter 33 already shows the shape a remote member would take: `stryke-aws`'s posture, not `App`'s.

Chapter 37 — Glossary

The terms below are the book's load-bearing vocabulary, each defined as this book uses it. Where a term has a broader meaning elsewhere, the definition here is the bus's.

allow-list. The dispatch discipline by which only verbs explicitly registered — in the engine's `COMMANDS` or through `ZGui.automation` — are callable, and every unrecognized verb is rejected with `no such verb`. The default answer to an unknown verb is `no`.

appShell. The shared frontend every app is built over, contributing the fifteen shared `appshell.*` verbs — terminal, files, palette, settings, shortcuts, theme, CRT, neon — that every app inherits without declaring them.

bridge. Layer 2: the correlated request/response substrate, with an attached event stream, that lets an in-flight script call back into an app and get the value into the VM. One substrate, used three ways — `call`, `get`, and `on`.

bus name. The short name that identifies an app on the bus — `zcite`, `zreq` — used by `App::open`, returned by `App::list`, and forming the socket filename `<app>.sock`.

call. The operation, and the frame kind, that invokes a verb with typed arguments and returns its value. The central operation of the bus; everything else supports it.

catalog. `GUI_SCRIPT_ACTIONS.md`, the machine-generated census of every app's live surface, re-taken on demand from the running apps. The authority on the exact verb set; a projection of the code, not prose about it.

core. The pure-Rust `*-core` engine each app is a thin shell over. Owns the app's verbs; its `COMMANDS` list is the authority on which verbs route straight to Rust.

event. An app-to-script notification — a named channel the app pushes down with `emit` and a script receives by subscribing with `on`. Answers “something just changed,” where a state query answers “what is true now.”

frame. One line of newline-delimited JSON on the wire. Seven kinds: `call`, `get`, `verbs`, `sub`, `unsub` from the client; `reply`, `event` from the server.

get. The operation, and the frame kind, that reads a state query — a read-only observation of the app's current condition, with no side effect.

handle. A live binding to one app: in-process, the host callback into the local surface; out-of-process, an open socket to one app's `.sock` file. Minted by `App::here` or `App::open`, VM-local, never a value that can cross to another app.

hybrid handler. The per-app `bus.rs` that routes each incoming call either to the engine in Rust (if the verb is in `COMMANDS`) or to the frontend via the dispatch shim (otherwise). The one app-specific decision the shared host delegates.

id. The correlation token on every request frame, matched by the reply's `id` and the event's `sub`. A conversation token, not a clock; what makes concurrency and event delivery safe.

in-process. The transport for a script running inside the app — a palette step or a hook — binding `App::here()` directly to the local surface through the host callback, with no socket.

manifest. The typed surface an app registers: its verbs, state queries, and events, with their parameter and return types. Read by a script as `verbs()`, stripped of function bodies; read by the palette's own picker; the single source both readers share.

out-of-process. The transport for a standalone `.stk` file driving a separately running app over its Unix socket. Adds one socket round trip and two JSON serializations over the in-process path.

pump. The explicit point — `App::pump()` — at which queued event callbacks run. Events never interrupt a running script; they queue and drain only where the script pumps, so the author always knows the exact set of places a callback can run.

star topology. The shape every multi-app run takes: the script at the center, each app a spoke, every cross-app value passing through the script's own variables. A consequence of the rule that a handle is not a value.

state query. A read-only named observation of an app's current condition — the selection, the active collection, the connection status — read with `get` and idempotent by contract.

surface. Layer 1: an app's typed manifest of verbs, state, and events, owned by the core and registered once on boot. Also, loosely, the whole set of what an app exposes to the bus.

verb. A named, typed, callable action an app declares — its id, parameters, return type, and runnable body. The unit of everything the bus does; an app's surface is mostly verbs.

wire. The newline-delimited JSON protocol carried by both transports. Deliberately small, legible, and closed: it grows by adding fields old readers ignore, never by adding frame kinds they would reject.

Chapter 38 – Tooling: The REPL and Completion in Practice

The reflective surface, spent as tools

Chapter 17 named two tools that fall out of an enumerable surface — a REPL and a completion source — and called them the surface's own reflectiveness, spent twice. This chapter builds both far enough to use, because a reader who has followed the design deserves the tools that make it livable, and because the way each is built is the last and clearest demonstration that everything the bus offers is a projection of one registration read many ways.

The REPL, built from two calls

A REPL for the bus is a loop over `verbs()` for discovery and `call` for execution, and its whole implementation is short because the surface does the work. It reads an app name, opens a handle, and then reads verb-and-argument lines, dispatching each and printing the decoded reply:

```
#!/usr/bin/env stryke
use App

val $name = prompt("app> ")
val $h = App::open($name)
p "connected to ${ $name } - ${\ scalar @ { $h->verbs()->{verbs} }} verbs. ? to list, q to
  ~ quit."

loop {
  val $line = prompt("${ $name }> ")
  if $line == "q" { break }
  if $line == "?" {
    for val $v (@ { $h->verbs()->{verbs} }) { p "  ${ $v->{id} }" }
    next
  }
  # a line is a verb id then a JSON argument object: request.send {"url":"..."}
  val ($verb, $json) = split_first($line, " ")
  val $res = try { $h->call($verb, parse_json($json ?? "{}")) }
    catch ($e) { p "! ${ $e }"; nil }
  if $res != nil { p pretty_json($res) }
}
```

Nothing in that loop is app-specific. It works against `zcite`, `zreq`, or an app added next year, because it drives every app through the same two calls, and its discovery — the `?` command — reads the live manifest, so the list of verbs it offers is always exactly what the connected app will answer to. The REPL is a database shell for a scriptable app, and it cost a loop.

A tracing REPL, for debugging

Chapter 23 named the verbose variant, and it is one added line: echo the frame you send and the frame you receive. In a REPL that already sends a `call` and receives a `reply`, a trace mode prints both around the decoded value:

```
if $trace {
  p "-> {\t\":"call\","verb\":"${ json_str($verb) },\"args\":"${ $json ?? "{}" }}"
}
val $res = $h->call($verb, parse_json($json ?? "{}"))
if $trace { p "<- ${ compact_json($res) }" }
p pretty_json($res)
```

Now the REPL is a probe: type a verb, watch the exact frame go out, watch the exact frame come back, read the value the module decoded from it. It is the fastest way to answer “is the app returning what I think it is?” because it puts a human at the exact seam where the script’s assumption meets the app’s reality — which is where Chapter 23 said most misbehaving-script questions are actually answered.

Shell completion, from `list` and `verbs`

The completion source is the second tool, and it is where the surface’s reflectiveness touches the shell the whole suite lives in. A completion function offers, after `App::open()`, the live set of running app names from `App::list()`, and after a handle’s `->call()`, the live set of verb ids from that app’s `verbs()`. The shape, as a completion callback:

```
# after App::open(" - complete the running app names
fn complete_app_name($prefix) {
  grep(App::list(), fn ($n) { starts_with($n, $prefix) })
}

# after $h->call(" - complete the verb ids of the app $h is bound to
fn complete_verb($h, $prefix) {
  map(grep(@{ $h->verbs()->{verbs} }, fn ($v) { starts_with($v->{id}, $prefix) })),
    fn ($v) { $v->{id} })
}
```

The completion is live in the strongest sense: it offers exactly the apps that are running and exactly the verbs they expose *at that moment*, not a static list baked into the completion at install time that drifts as apps start, stop, and gain verbs. A script author tab-completing against `App::open("z` sees only the apps actually up; tab-completing a verb after opening `zreq` sees only `zreq`’s current verbs. The completion cannot go stale because it is not a copy of the surface — it is the surface, queried at the moment of the tab.

Why neither tool is a feature of the bus

It matters that neither the REPL nor the completion is a capability the bus added; both are the surface’s own enumerability, spent once each. The bus did not ship a REPL and a completion engine; it shipped an enumerable surface, and a REPL and a completion are what an enumerable surface makes almost free. This is the deepest form of the “one declaration, many readers” property the book has traced from Chapter 3: the same registration that makes an app scriptable makes its palette picker richer, makes its verbs discoverable, makes a catalog generatable, makes a REPL possible, and makes a completion live — six readers of one declaration, none of which the app author wrote, all of which the app author gets for declaring the surface once. A capability whose tooling falls out of its data model rather than being built beside it is a capability that was designed at the right layer, and the REPL and the completion are the proof, because they exist without a line of code that knows any app’s name.

The tools compound

A last observation closes the practical arc. The tools do not merely coexist; they compound, because each is built from the same seven calls and so each works inside the others. The completion completes the verbs the REPL calls; the REPL, in trace mode, shows the frames the debugging chapter reads; the catalog the versioning chapter diffs is the same enumeration the completion offers live; and a helper library written against the seven calls, per Chapter 32, is callable from a script, a REPL session, or a maintenance run without change. The uniformity that let one crate host eleven apps and one module drive them all is the same uniformity that lets the tools built on top interlock — one surface, one wire, one vocabulary, and every tool a different reading of it. That is the whole system in its final shape: not a stack of features, but one programmable surface and the many ways a script, a shell, a REPL, and a generator each read it — every one of them handed, at last, a caller.

Chapter 39 – A Complete Session, Narrated

From cold shell to finished run

The book has shown the bus in pieces — a call here, a recipe there, a tool built to illustrate a property. This closing chapter runs one session whole, from a cold terminal to a finished cross-app result, narrating each step in the vocabulary the earlier chapters built, so that a reader sees the mechanisms not as separate chapters but as the single continuous experience they add up to in use. The task is concrete and ordinary: gather the current selection from the reference manager, look each item up against a live API, file the ones that resolve into a new collection, and mail a summary — four apps, one file, one run.

Discovery: what is even running?

The session begins the way most do, by asking the machine what it is working with. At a cold prompt, the first move is `App::list()`, and its answer is the directory scan of Chapter 6 rendered as a fact:

```
val @up = @{ App::list() }
p "running: ${\ join(", ", @up) }"
# running: zcite, zreq, zemail, zwire, zoffice
```

Four of the apps the task needs are up; the fifth it will not need. This is the undialed-call guard of Chapter 29 used prophylactically — check the list before opening, so a target that happens to be down is a skipped step rather than a death at `connect`. The script has learned, in one call, exactly which apps it can drive, from the live socket directory and nothing else.

Introspection: does the app still call it that?

The second move is the discover-before-call pattern of Chapter 34. Rather than trust its memory of a verb id, the script asks the app, reading the manifest of Chapter 3 through the `verbs()` of Chapter 5:

```
val $cite = App::open("zcite")
if !has_verb($cite, "library.search") { die "zcite surface changed" }
```

The `has_verb` helper of Chapter 32 is built from `verbs()` alone, works against any app, and turns “I hope the verb is still named that” into “the manifest says it is, or the script stops now with a clear reason.” This is the surface documenting itself, per Chapter 17: the script consults the running app, not a wiki, and the answer cannot be stale because it is the same registration the app will honor a moment later.

The read: state, not a verb

Now the work. The script reads the current selection — a *state query* of Chapter 27, not a verb, because the selection is a live condition the app maintains, and reading it twice would return the same answer with no side effect:

```
val @sel = @{ $cite->get("selection") }
p "selected ${\ scalar @sel} items"
```

`get("selection")` crosses the socket as the `get` frame of Chapter 28 and comes back as a stryke array of maps, projected from JSON by the value model of Chapter 13. The selection lived behind a window the old palette could not read; now it is a variable the script walks.

The cross-app loop, with failure tolerance

The center of the run is a loop that leaves one app and enters another once per item — the star topology of Chapter 13, every value passing through the script — with the try-per-item tolerance of Chapter 34 so one bad lookup does not end the run:

```
val $req = App::open("zreq")
val $cite2 = $cite # same handle; the hub stays the script
val @filed = @{}

$cite->call("collection.add", %{ name => "verified" })

for val $it (@{ @sel }) {
  val $doi = $it->{doi}
  if !$doi { next }
  val $res = try { $req->call("request.send",
    %{ url => "https://api.crossref.org/works/${ $doi }" }) }
    catch ($e) { p "lookup failed for ${ $doi }: ${ $e }"; nil }
  if $res && $res->{status} == 200 {
    $cite->call("collection.add_item", %{ collection => "verified", item => $it->{id}
    ~ })
    push @filed, $it->{title}
  }
}
p "verified and filed ${\ scalar @filed} items"
```

Read the seams the way Chapter 16 taught. `$it->{doi}` leaves `zcite`'s selection; the crossref URL built from it enters `zreq`'s `request.send`; the response's status decides whether `$it->{id}` goes back into `zcite`'s `collection.add_item`. Two apps, and the only place their data meets is the loop body — a program the user wrote and can read top to bottom. The `try` turns a failed lookup into a logged skip, per the failure model of Chapter 14, so a network hiccup on one DOI costs one item, not the run.

The finish: a summary, mailed

The run ends by crossing to a third app, `zemail`, to send the summary — the value that was computed in the loop leaving the script and becoming a message body:

```
val $mail = App::open("zemail")
$mail->call("message.send", %{
  to      => "me@example.com",
  subject => "verified ${\ scalar @filed} references",
  body    => "Filed into 'verified':\n" + join("\n", map(@filed, fn ($t) { "  - ${ $t }"
  ~ })))
})
p "summary sent; run complete"
```

`message.send` is `zemail`'s own verb, and the body is a string the script built from the loop's results. No app knew about the others; the reference manager never heard of the API client, which never heard of the mail client. Each answered the script, and the script was the one place all three met.

What just happened, in the book's terms

Step back and name the mechanisms the session spent, because their sum is the whole book. `App::list` read the socket directory (Chapter 6). `App::open` dialed three sockets, each a handle that is a connection and not a value (Chapters 5, 13). `verbs()` and `has_verb` introspected the live manifest (Chapters 3, 17, 32). `get("selection")` read state, not a verb (Chapter 27). The loop stayed a star with the script at the hub (Chapter 13). `request.send`, `collection.add_item`, and `message.send` were called with typed arguments and returned values into variables (Chapters 2, 4). A `try` made a failure a recorded skip rather than a death (Chapter 14). Every value that crossed between apps crossed through the script, projected from JSON and back at each boundary (Chapter 13). And underneath all of it, the same newline of JSON carried every frame, correlated by the same `id`, over the same shared crate serving each app's one socket (Chapters 6, 7, 31).

That is the sentence the book opened on, executed once: one language drove every app the task needed, by name, and got each answer back into a variable the next call could use. The apps were always sets of functions waiting to be called; the session is what it looks like when they are finally called — from one file, one terminal, one run, by a script the bus handed a caller.

Chapter 40 – The Bus in the Shell

The `.stk` file is a first-class citizen of the terminal

The out-of-process front-end is a standalone `.stk` file, and calling it “standalone” undersells where it lives: it lives in the terminal, alongside every other executable the shell knows, and it composes with them the way any other program does. A bus script has a shebang — `#!/usr/bin/env stryke` — so it is `chmod-executable` and runnable by name; it reads arguments, it reads `stdin`, it writes `stdout` and `stderr`, and it returns an exit status. None of that is bus machinery; it is `stryke` being an ordinary member of the shell environment, and it matters because it means a bus script is not a special artifact that runs in a special way. It is a program, and the shell composes programs. The bus’s cross-app reach and the shell’s composition are two powers that multiply: a script that drives four apps is also a program a pipeline can feed and a scheduler can run.

Arguments and exit status: the shell’s contract

A bus script that takes arguments and returns a status is a script the shell can drive, and the maintenance and test recipes leaned on exactly this without naming it. A script that `dies` on failure exits non-zero, which a scheduler reads as “this run failed” and a `ff` chain reads as “stop here”; a script that finishes cleanly exits zero, which the same constructs read as success. The API regression runner of Chapter 35 ended in `die` `"${ $fail } assertion(s) failed"` for precisely this reason: its exit status is its verdict, and a CI job runs it as `stk smoke.stk || alert`, needing to know nothing about the bus to act on the result. The bus script speaks the one protocol every program in the terminal speaks — argument in, status out — so it slots into the shell’s world without translation.

Piping data in and out

Because a bus script reads `stdin` and writes `stdout`, it composes with the rest of the terminal by pipe. A script that reads DOIs on `stdin` and files each into the reference manager is fed by anything that produces DOIs — a `grep`, a `cat`, another script — and a script that writes its results to `stdout` feeds anything downstream:

```
#!/usr/bin/env stryke
use App
val $cite = App::open("zcite")
for val $doi (@{ lines(stdin()) }) {           # one DOI per input line
    val $item = try { $cite->call("item.add", %{ doi => trim($doi) }) } catch ($e) { nil }
    if $item { p $item->{id} }                  # emit the new id to stdout
}
```

Run as `cat dois.txt | stk file-dois.stk > new-ids.txt`, this bus script is a filter — DOIs in, ids out — and the reference manager is a stage in a shell pipeline. That is a strange and useful sentence: a GUI reference manager became a `grep`-able stage in a terminal pipe, because the bus made its `item.add` callable and `stryke` made the caller a filter. The composition is the shell’s; the reach into the app is the bus’s; and neither had to know about the other for the two to multiply.

Scheduling and the unattended run

A bus script that returns a status and needs no terminal is a script a scheduler runs, and the reporting and maintenance recipes of Chapter 30 were written for exactly this life. A cron entry or a systemd timer runs `stk maintenance.stk`, which checks `App::list()` for the apps that are up, does what it can, records the rest, and exits non-zero if anything failed. The unattended run leans on every part of the failure model — the list check that skips a down app, the try-per-item that records rather than aborts, the exit status that reports the verdict — and it leans on the at-rest cost of the bus being nil, per Chapter 24, so that apps carrying the bus pay nothing for the hours between scheduled runs. A scheduled bus script is the automation the book has been building toward at its most self-effacing: it drives the whole suite and no one is watching, and it is trusted to because it fails loudly and finishes cleanly.

The bus and `zsh`

The terminal the bus scripts run in is itself part of the stack — the shell is the IDE, the build system, the orchestrator — and the bus is a citizen of it rather than a thing beside it. A bus script is invoked from the shell, completes against the live surface through the completion of Chapter 38, and returns to the shell; and the shell's own history, aliases, and composition apply to it as to any command. The point is not that the bus integrates with the shell — it is that there is no seam to integrate across, because a bus script is a shell program that happens to reach into GUI apps. The suite's discipline that everything is a set of functions to be called reaches its widest form here: the shell calls the script, the script calls the apps, and the whole chain — keystroke to shell to script to socket to verb to engine and back — is one continuous set of calls, each handed its caller by the layer above. The bus did not build a bridge to the terminal; it noticed the terminal was already the place the caller lives, and put the apps within its reach.

The last sentence

The book began on one sentence and has spent forty chapters earning it, so it ends by saying it plainly one more time, now with every word behind it. One language should be able to drive every app you own, by name, and get an answer back. The language is `stryke`. The apps are the suite. The driving is a typed verb, called with typed arguments, over a host callback or a newline of JSON. The answer back is the return value, landing in a variable the next call can use. And the reason the pieces that make the sentence true could be small — a manifest, a bridge, a module, a socket, a shell script — is that the architecture had already turned every app into a set of functions waiting to be called. The bus only had to hand them a caller, and this book only had to show that it did.